



UNIVERSITY OF PISA

DEPARTMENT OF INFORMATION ENGINEERING

Large-Scale and Multi-Structured Databases

WineAdvisor

Work group:

Bertei Valentina

Bertelà Marta

Tarchi Francesco

ACADEMIC YEAR 2024/2025

Contents

1	Application Manual	5
1.1	Application Highlights	5
1.2	User Manual	6
1.2.1	Basic User	6
1.2.2	Winery User	7
1.2.3	Administrators	7
2	Design Overview	9
2.1	Main Actors	9
2.2	Requirements	9
2.2.1	Functional Requirements	9
2.2.2	Non-functional Requirements	11
2.3	Use Case Diagram	12
2.4	UML Class Diagram	12
3	Data Modeling and Structure	15
3.1	Dataset Creation	15
3.1.1	Web APIs	15
3.1.2	Scraping	16
3.1.3	Building Documents	16
3.2	Scripts and functions for dataset assembly	17
3.3	Databases	18
3.3.1	Volume Considerations	18
3.3.2	MongoDB	19
3.3.2.1	MongoDB Collections	19
3.3.3	Neo4j	36
3.3.3.1	Entities	37
3.3.3.2	Relationships	37
4	Implementation	41
4.1	Implementation Frameworks	41
4.2	Project Structure and Packages Organization	41
4.2.1	<code>com.wineadvisor.wineadvisor.config</code>	42
4.2.2	<code>com.wineadvisor.wineadvisor.controller</code>	42

4.2.3	<code>com.wineadvisor.wineadvisor.model</code>	43
4.2.4	<code>com.wineadvisor.wineadvisor.service</code>	45
4.2.5	<code>com.wineadvisor.wineadvisor.repository</code>	46
4.2.6	<code>com.wineadvisor.wineadvisor.DTO</code>	48
4.2.7	<code>com.wineadvisor.wineadvisor.exception</code>	48
4.3	MongoDB Operations	51
4.3.1	CRUD operations	51
4.3.2	MongoDB Aggregations for Analytics	58
4.4	Neo4j Graph Queries	77
4.4.1	CRUD operations	77
4.4.2	Relevant Operations	78
4.5	Admin Operations	84
5	Database Choices	89
5.1	Introduction to VM Organization	89
5.2	Indexes	89
5.2.1	MongoDB - reviews: <code>index(wine_id.id ASC)</code>	89
5.2.2	MongoDB - reviews: <code>index(user_id.username ASC, rating DESC, created_at DESC)</code>	91
5.2.3	MongoDB - wines: <code>index(style.name ASC, statistics.ratings_average DESC, statistics.ratings_count DESC)</code>	92
5.2.4	MongoDB - wines: <code>index(style.name ASC, created_at DESC)</code>	93
5.2.5	Neo4j indexes considerations	93
5.3	Replication	93
5.3.1	MongoDB Replication	93
5.3.2	Consideration on CAP Theorem	93
5.4	Sharding	94

Chapter 1

Application Manual

This guide explores the features and vision behind WineAdvisor, a web platform developed during the Large-Scale and Multi-Structured Databases course. WineAdvisor is designed for wine lovers, offering a platform to discover, review and share wines from all over the world. Users can search for wines based on various criteria, such as region, grape variety and price. The application also features a personalized recommendation system and social features such as following other users and exploring their favorites. WineAdvisor aims at enhancing the wine experience through statistics, tailored suggestions, and support for wine tourism.

The code of the project is available at
<https://github.com/francetarchi/LargeProject>.

1.1 Application Highlights

WineAdvisor integrates a range of features designed to provide users with a rich, personalized, and socially engaging wine experience. Below are the key functionalities that characterize the application:

- **Advanced Search Capabilities:** Users can search for wines based on a variety of criteria, including price range, country and region of origin, grape variety and wine type. This allows for highly targeted and meaningful exploration.
- **User Reviews and Favorites:** Each user has the ability to rate and review wines, as well as curate a personal list of favorites, which can be accessed and updated at any time.
- **Personalized Recommendations:** The application offers wine suggestions based on user preferences, previous reviews, and newly released labels, helping users discover products aligned with their tastes.
- **Wine Statistics and Trends:** WineAdvisor presents insights such as review trends over specific time periods, the most appreciated wines in a given area,

and top-rated wines in terms of quality-to-price ratio.

- **Social Interaction:** Users can follow others and be followed in return, gaining access to their reviews and favorite selections. This fosters a community-driven approach to wine discovery.
- **Business Accounts:** Wine producers and distributors can register as companies, enabling them to manage their wine catalogues directly by uploading, editing, or removing products.

1.2 User Manual

When accessing the site, users are presented with options to log in or register.

To fully access and utilize the features of the application, users are required to create an account and log in. Without registering, users will not be able to interact with the platform or benefit from its functionalities.

The application supports three main types of users: **Basic Users**, **Winery Users**, and **Administrators**. **Basic Users** represent regular customers who can search for wines, write reviews, interact with other users, and perform additional actions described in detail in the dedicated user manual chapter. **Winery Users** correspond to wine businesses and have access to a different set of functionalities, such as managing their own wine catalog by adding, editing, or removing products. Some features available to Basic Users are not accessible to Winery Users, and vice versa, reflecting the distinct roles they hold within the platform. Finally, **Administrators** have full access to all functionalities and are responsible for maintaining the integrity and performance of the platform.

1.2.1 Basic User

After registering and logging in, Basic Users gain access to the core functionalities of WineAdvisor. They are welcomed with a personalized homepage, featuring a dynamic feed populated by posts and reviews from the users they follow.

Basic Users can:

- Search for wines using various filters such as region, grape variety, price, type;
- Explore wine suggestions based on their preferences, review history, and new releases;
- Write and post reviews about the wines they have tasted;
- Add wines to their list of favorites for easy access;
- Like or dislike reviews posted by other users;
- Search for other users or winery profiles and view their public activities;

- Follow or unfollow users and wineries to personalize their feed and discover new wines.

1.2.2 Winery User

Winery Users represent wine-producing businesses that manage their own digital presence on the platform. After creating a winery account and logging in, these users gain access to a dedicated interface that enables them to curate and update information about their products and brand. Winery Users can:

- Upload and manage their wine catalog, including the ability to add, edit, or remove wines;
- Specify different vintages for each wine and update related information;
- Create a gallery of images showcasing their winery, to enhance visibility and brand identity;
- View reviews and feedback posted by Basic Users about their wines.

Unlike Basic Users, Winery Users do not have access to social features such as writing reviews or following other users. Their role is focused on product and brand management within the WineAdvisor system.

1.2.3 Administrators

Administrators hold the highest level of privileges within the application. Their role encompasses full access to all functionalities available to both Basic Users and Winery Users. This means they can explore the platform as a regular user, interact with content, and manage wine listings as a winery. In addition to these capabilities, Administrators are entrusted with complete control over the database and its resources. They can create, modify, or delete any data, including user accounts, wine entries, reviews, and statistics. This level of access allows them to oversee the correct functioning of the system, perform maintenance operations, and ensure that the platform remains secure, up-to-date, and aligned with its intended goals.

Chapter 2

Design Overview

In this chapter, we take a closer look at how the system is built, focusing on its main components and how they work together. Understanding this structure helps to make sense of how the application works and how it can grow or be improved in the future.

2.1 Main Actors

The WineAdvisor system involves three main types of actors, each playing a distinct role within the platform:

- **Basic Users:** Regular users who interact with the application by searching for wines, posting reviews, exploring suggestions, and engaging with other users through a social feed. They represent the core audience of the platform.
- **Winery Users:** These accounts are managed by wine businesses and producers. Their primary role is to upload, manage, and promote their own wine catalog. Winery Users can add new wines and vintages, modify product details, and showcase their winery through images and descriptions. Unlike Basic Users, they cannot write reviews or engage in social features as regular users do.
- **Administrators:** Admins have full control over the system. They can access and manage all resources available to both Basic and Winery Users, and additionally have the ability to edit or remove any content in the database. Their role ensures the proper functioning and maintenance of the entire platform.

2.2 Requirements

2.2.1 Functional Requirements

This section outlines the main functionalities that each type of user can access within the WineAdvisor platform. These requirements are essential to ensure the system

supports the intended user experience and behaviors.

Unlogged Users

Unlogged users should be able to:

- Register and create an account to become a Basic or Winery User.
- Log in with existing credentials in order to access the platform.

Basic Users

After registering and logging in, Basic Users should be able to:

- Access a personalized homepage/feed with updates from followed users and wine suggestions.
- Search for wines by name, region, grape variety, price, and more.
- Review wines and assign a rating and optional description.
- Like or dislike reviews from other users.
- Add wines to their list of favorites.
- Follow other users and view their profiles and reviews.
- Explore wine recommendations based on their tastes and past activity.

Winery Users

Winery Users should be able to:

- Manage their company profile and update winery information.
- Upload new wines and vintages, including all relevant details.
- Edit or remove wines they own from the catalog.
- Add an image gallery of their winery.
- View reviews and statistics related to their wines.
- Search for other wines or users for reference or comparison.
- Note: Winery Users cannot post reviews or interact socially like Basic Users.

Administrators

Administrators have full access to all platform functionalities. They should be able to:

- Perform all actions available to both Basic and Winery Users.
- Monitor, edit, and remove any content within the system, including wines, users, and reviews.
- Manage platform statistics and initiate scheduled updates or data recalculations.
- Maintain data integrity and oversee the general health of the system.

2.2.2 Non-functional Requirements

These requirements ensure the system's quality in terms of performance, reliability, maintainability, and security. While not directly describing user-facing features, they are fundamental for the stability and long-term success of the platform.

- **Web-Based Architecture:** The application is implemented as a web service, providing RESTful endpoints to enable future integration with graphical interfaces or mobile clients.
- **Resilience and Fault Tolerance:** The architecture minimizes single points of failure to enhance the system's robustness. Failures in non-critical components should not compromise core functionalities.
- **High Availability:** The system is designed to be accessible with minimal downtime, even if data presented (e.g., statistics or ranking lists) is slightly outdated.
- **Modular and Object-Oriented Design:** Code is developed using an object-oriented paradigm, promoting separation of concerns, code reusability, and ease of maintenance.
- **Graceful Degradation:** The system should be able to tolerate limited data loss in non-critical areas, ensuring continuity of service even under exceptional conditions.
- **Password Security:** User passwords are securely stored using the *BCrypt* hashing algorithm, which incorporates salting and an adaptive workload factor to protect against brute-force attacks and ensure strong encryption over time.
- **Future-Proofing:** The system is designed with extensibility in mind, allowing for the addition of new roles, features, and data sources without requiring major refactoring.

2.3 Use Case Diagram

This section presents the *Use Case Diagram* that models the interactions between the main actors of the WineAdvisor system and the functionalities they can access. The diagram helps to visualize the system from a user-centric perspective, highlighting the roles of each actor and their respective operations within the platform.

2.4 UML Class Diagram

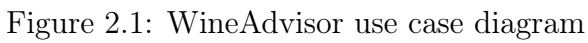


Figure 2.1: WineAdvisor use case diagram

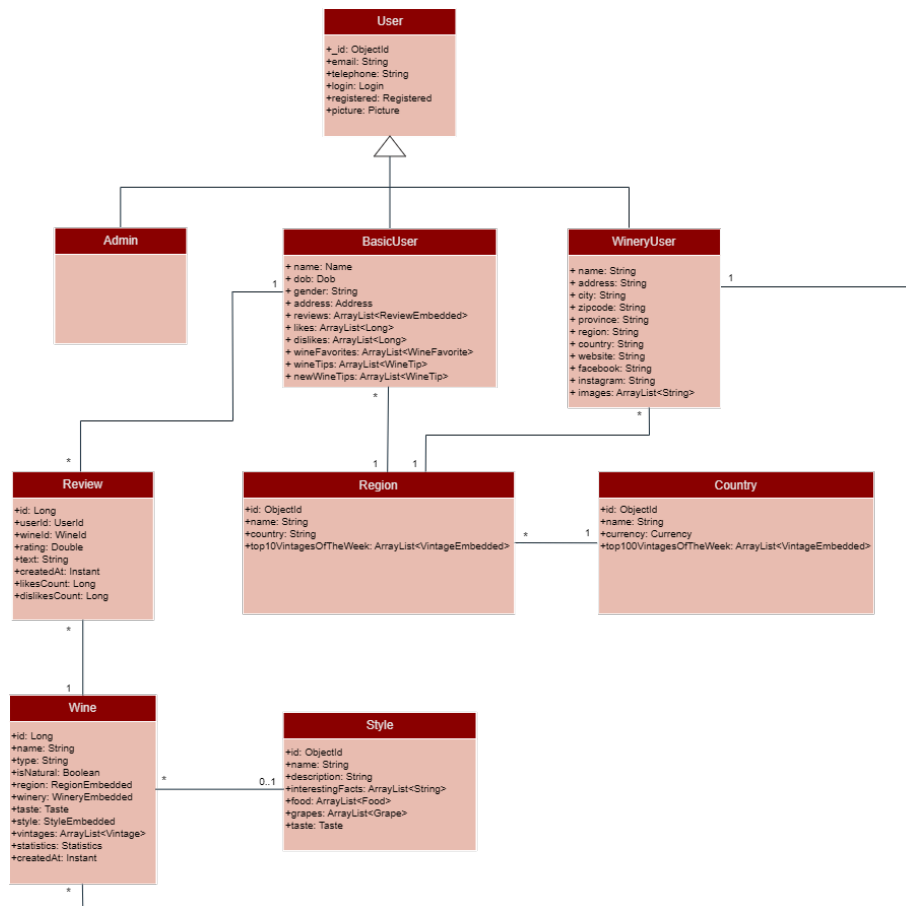


Figure 2.2: WineAdvisor class diagram

Chapter 3

Data Modeling and Structure

3.1 Dataset Creation

In accordance with the principle of data variety, our dataset was assembled from a combination of multiple structured sources, public repositories, and scraped content. To ensure a diverse and representative dataset, we relied on multiple sources and techniques, all oriented toward maximizing data realism. The core of our dataset was built using data retrieved from three main sources: **Vivino.com**, **Bereilvino.it**, and **RandomUser.me**. Each of these sources served a specific purpose within the dataset creation pipeline and contributed unique information.

3.1.1 Web APIs

Vivino API

We used Vivino’s public (but unofficial) API to retrieve structured data related to wines and their vintages. The API queries returned arrays of matches, where each match corresponded to a specific vintage of a wine. Within each vintage object, we accessed keys such as `price`, `prices`, and a nested `wine` object, which contained all relevant information about the wine itself. This structure suggested a relational backend model, where vintages and wines are stored separately and cross-referenced. Statistical attributes such as average rating and number of reviews were provided per vintage, not per wine, emphasizing the need to separate these two entities in our modeling.

RandomUser API

To simulate a realistic user base for our application, we generated synthetic user data through the `randomuser.me` API. This allowed us to populate our system with diverse user profiles while avoiding any privacy concerns or the need for real user data.

3.1.2 Scraping

Bereilvino.it

Since no public API was available, we resorted to web scraping to extract wine information from `bereilvino.it`, a popular Italian wine portal. The scraped data included details such as wine names, producers, regions, and occasionally pricing or awards. This information was semi-structured and required post-processing and normalization before being integrated into our dataset. We made sure to respect the website's structure and implemented rate limiting to minimize server load during the extraction process.

3.1.3 Building Documents

Reviews

Our review dataset was obtained primarily from Vivino. Each review was associated with a specific vintage, not directly with the wine itself. To build consistent review documents, we extracted key fields such as rating (on a scale from 0 to 5), review text, timestamp, and the corresponding user and vintage identifiers.

Users

User profiles were generated using two main strategies. A portion of users was created by leveraging the `randomuser.me` API, which provided realistic personal information such as names, surnames, emails, birthdates, and nationalities. For the remaining users, we employed the **Faker** Python library to synthetically generate similar fields. Where available, we preserved the original usernames found in the Vivino reviews, enriching those profiles with additional personal details. To ensure secure authentication, passwords were hashed using the `bcrypt` algorithm.

Wineries

Wineries were extracted from `bereilvino.it` using web scraping techniques. Each winery document included essential attributes such as the name, region, and country. Since wineries are also a specific type of account within the system, login credentials and access-related fields (e.g., email, password) were synthetically generated using the **Faker** Python library. Passwords were hashed using the `bcrypt` algorithm to ensure secure authentication.

Wines

Wine documents were built by combining information from Vivino. Each document contained fields such as name, type, style, taste, winery reference, and a nested list of vintages. Each vintage stored year-specific data. This structure ensured that both general wine attributes and vintage-specific data could be efficiently queried and displayed.

Styles

Wine styles were originally embedded within the *Wines* documents. To improve data normalization and reusability, these styles were extracted and modeled as a separate entity. The *Styles* documents include the style name, description, and typical characteristics associated with each style. This separation allows for easier querying and classification of wines by their style attributes across the dataset.

Regions and Countries

The *Regions* and *Countries* documents were manually curated to ensure accuracy and consistency across the dataset. These entities provide standardized geographic classifications used to organize wineries and wines. Each region document includes the region name and its associated country, while each country document contains essential information such as the country name and currency.

3.2 Scripts and functions for dataset assembly

To prepare a consistent and enriched dataset, we developed a suite of Python scripts performing data cleaning, transformation, and integration tasks across multiple source files. Key operations included:

- Populating flavor profiles and image links, including generating a set of random images saved in the database, referenced by an `image` field where needed.
- Standardizing review documents by renaming `note` to `review_text`, removing redundant fields such as `language`, and adding `wine_id` to reviews for better linkage.
- Reorganizing statistical data by moving wine-related statistics from individual vintages to the parent wine document.
- Cleaning style information by removing unnecessary fields.
- Merging and deduplicating wineries data, assigning unique IDs, and updating winery references in wines accordingly.
- Embedding user data (username and profile picture) directly into wine reviews to reduce relational queries.
- Transforming review relations by replacing `user_id` and `wine_id` fields with embedded documents containing relevant user and wine details.
- Augmenting user profiles with samples of their recent reviews.
- Associating the three most recent reviews with each vintage, instead of the wine as a whole.

- Normalizing geographic data by replacing sub-regions with official Italian regions in wine documents.
- Replacing numeric `type_id` fields in wines with descriptive strings (e.g., "rosso", "bianco").
- Separating fortified or aromatized wines into a distinct category.
- Moving the `image` field from the wine level to each vintage document.
- Cleaning country data by removing unnecessary fields.
- Standardizing user fields: renaming `phone` to `telephone` and `state` to `region`, removing unused fields like `timezone` and `coordinates`.
- Localizing user data to Italian profiles with assistance from GPT-generated formatting.
- Removing redundant country information from individual wine documents.
- Proposing the introduction of `Region` and `Country` entities within the GraphDB to enable geographically contextualized recommendations for users and wineries.
- Adding `likes` and `dislikes` arrays to each user to efficiently manage user reactions to reviews and avoid duplication of likes or dislikes.

This comprehensive preprocessing pipeline ensured data consistency, enriched document contents, and optimized the dataset for both querying and analytical use cases.

3.3 Databases

Our application relies on **MongoDB** and **Neo4j**. In this chapter, we delve into the rationale behind this selection, the methods used to store data, and the thoughtful techniques we applied.

3.3.1 Volume Considerations

Action	% Users	Daily Occurrences	MongoDB Ops	Neo4j Ops	Description
Wine Search	40%	10,000	1	0	Query for searching wines
View Wine Page	50%	15,000	1	1	Viewing wine details
Write Review	10%	1,500	2	0	Creating a review + updating stats
Like Review	20%	5,000	1	0	Like or dislike a review
Winery Search	30%	5,000	1	1	Searching wineries
View Winery Page	25%	7,500	1	1	Viewing winery details
Totals			39,000	23,500	

Table 3.1: Estimated Daily Requests per Action

Based on an estimated user base of approximately 5,000 users, we assume that 20% are daily active users (DAU), and an additional 10% access the platform occasionally. With an average of 8.8 actions per active user per day, this results in an estimated 44,000 user actions daily. The following table provides a breakdown of the main user interactions, along with the estimated number of daily occurrences and the corresponding operations performed on MongoDB and Neo4j.

3.3.2 MongoDB

Although MongoDB was a required technology for the project, we would have chosen it regardless due to the advantages offered by its document-oriented approach. The ability to embed related information within a single document allows us to avoid complex joins, leading to simpler queries and better runtime performance. Additionally, MongoDB's horizontal scalability makes it a solid choice for applications expected to grow over time, thanks to efficient indexing and a robust aggregation pipeline for processing large datasets.

3.3.2.1 MongoDB Collections

These are the following MongoDB collections used in our application:

- Reviews
- Users
- Wines
- Wineries
- Admins
- Regions
- Countries
- Styles
- Counters
- TopVintagesByOurQopType
- TopVintagesByQopType
- TopVintagesRatingsType
- TopWineriesRatings
- TopWinesRatingsType

Reviews Collection

Here is an example of a document stored in this collection:

```
{
  "_id": 1,
  "user_id": {
    "username": "silverelephant535",
    "thumbnail": "https://randomuser.me/api/portraits/thumb/men/98.jpg"
  },
  "wine_id": {
    "id": 542,
    "name": "Oracolo Veneto Rosso",
    "year": 2000,
    "image": "https://i.postimg.cc/76rp4DV8/wine19.png"
  },
  "rating": 4.0,
  "text": "Un vino ricco e strutturato, con un equilibrio perfetto tra tannini e frutta matura, ideale con piatti di carne alla griglia.",
  "created_at": {
    "$date": "2024-12-10T17:26:49.316Z"
  },
  "likes_count": 180,
  "dislikes_count": 268
}
```

Users Collection

Here is an example of a document stored in this collection:

```
{
  "_id": {
    "$oid": "6839380fcb18a722a38c545b"
  },
  "gender": "female",
  "name": {
    "title": "Miss.",
    "first": "Michela",
    "last": "Gianetti"
  },
  "address": {
    "street": {
      "number": "49",
      "name": "Viale Gustavo"
    },
    "city": "Gorizia",
  }
```

```
    "region": "Friuli-Venezia Giulia",
    "country": "Italia",
    "postcode": "34170"
  },
  "email": "michela.gianetti@example.com",
  "login": {
    "username": "crazyduck840",
    "password":
      "$2b$12$.iJSeJyL1M7.m7Vg/QR/Se1Px33jtISjsRvhAlbixgNEvb3n8xqkq"
  },
  "dob": {
    "date": {
      "$date": "1981-04-28T03:32:29.035Z"
    },
    "age": 43
  },
  "registered": {
    "date": {
      "$date": "2020-02-14T19:45:21.187Z"
    },
    "age": 4
  },
  "telephone": "+39 327 9756577",
  "picture": {
    "large": "https://randomuser.me/api/portraits/women/95.jpg",
    "medium": "https://randomuser.me/api/portraits/med/women/95.jpg",
    "thumbnail":
      "https://randomuser.me/api/portraits/thumb/women/95.jpg"
  },
  "reviews": [
    {
      "review_id": 16381,
      "user_id": {
        "username": "crazyduck840",
        "thumbnail":
          "https://randomuser.me/api/portraits/thumb/women/95.jpg"
      },
      "wine_id": {
        "id": 4461,
        "name": "Ovello Barbaresco",
        "year": 2020,
        "image": "https://i.postimg.cc/BZLhqt6r/wine20.png"
      }
    },
  ]
}
```

```
"rating": 4.6,
"text": "Superbo e umile.",
"created_at": {
  "$date": "2023-10-30T10:22:47.000Z"
},
"likes_count": 90,
"dislikes_count": 399
},
{
  "review_id": 3602,
  "user_id": {
    "username": "crazyduck840",
    "thumbnail":
      "https://randomuser.me/api/portraits/thumb/women/95.jpg"
  },
  "wine_id": {
    "id": 201,
    "name": "Pinot Nero",
    "year": 2021,
    "image": "https://i.postimg.cc/76rp4DV8/wine19.png"
  },
  "rating": 4.0,
  "text": "Ciliegie, fragole, more mature.",
  "created_at": {
    "$date": "2022-04-15T21:37:06.000Z"
  },
  "likes_count": 1,
  "dislikes_count": 303
},
{
  "review_id": 8323,
  "user_id": {
    "username": "crazyduck840",
    "thumbnail":
      "https://randomuser.me/api/portraits/thumb/women/95.jpg"
  },
  "wine_id": {
    "id": 1775,
    "name": "Faius",
    "year": 2021,
    "image": "https://i.postimg.cc/44SmQSNk/wine12.png"
  },
  "rating": 4.0,
```

```
        "text": "Gran vino, con finale persistente.",
        "created_at": {
            "$date": "2021-08-21T18:55:57.000Z"
        },
        "likes_count": 1,
        "dislikes_count": 17
    }
],
"likes": [
    21,
    25,
    31,
    64,
    68,
    85,
    90
],
"dislikes": [
    8,
    19,
    107,
    117,
    128,
    143
],
"wine_favorites": [],
"wine_tips": [],
"new_wine_tips": []
}
```

As shown, each user document includes their three most recent reviews. This list is updated in real time when a new review is submitted or an existing one is deleted: newly submitted reviews are immediately added to the top of the list, and deleted ones are instantly removed. However, the operation that ensures the array always contains the three most recent reviews in case of inconsistencies (e.g., after deletion) is scheduled to run once per day. This approach enables us to display a summary of a user's reviews directly on their profile page, whether viewed by themselves or others, without needing to query a separate collection. In this way, users can quickly get an overview of someone else's interests. The arrays `likes` and `dislikes` contain the IDs of reviews that the user has liked or disliked, respectively. The `wine_favorites` array lists the user's favorite wines, which serve as the basis for generating personalized recommendations found in the `wine_tips` and `new_wine_tips` arrays below.

Wines Collection

Here is an example of a document stored in this collection:

```
{
  "_id": 1,
  "name": "Grandi Annate Sangiovese",
  "type": "rosso",
  "is_natural": false,
  "region": {
    "name": "Toscana",
    "country": {
      "name": "Italia",
      "currency": {
        "code": "EUR",
        "name": "Euro",
        "prefix": "€"
      }
    }
  },
  "winery": {
    "name": "Avignonesi",
    "username": "avignonesi2572",
    "thumbnail":
      "https://i.postimg.cc/LX6F6wtH/winery112.jpg"
  },
  "taste": {
    "acidity": 3.7327719,
    "fizziness": null,
    "intensity": 3.4893339,
    "sweetness": 1.5484701,
    "tannin": 3.5251772
  },
  "style": {
    "name": "Vino Nobile di Montepulciano",
    "description": "You may not be familiar with Vino Nobile
di Montepulciano, but you should be. Despite its long name,
it makes delicious wines filled with preserved
red cherry flavors. ...",
    "interesting_facts": [
      "Vino Nobile di Montepulciano was one of the original
four DOCGs created in 1980.",
      "By law, the wines must be aged for 2 years to be called
Vino Nobile and 3 years to qualify as Riserva.",
      "The area of Montepulciano is warmer than
the rest of Tuscany.",
    ]
  }
}
```



```
        "Vino Nobile di Montepulciano must contain at least 70%
        Prugnolo Gentile (aka Sangiovese).",
    ],
    "food": [
        {
            "name": "Manzo",
            "image": "https://i.postimg.cc/mDbRbCNG/manzo.jpg"
        },
        {
            "name": "Cacciagione (cervo, camoscio)",
            "image":
                "https://i.postimg.cc/Vshyh3Tw/cacciagione.jpg"
        }
    ],
    "grapes": [
        {
            "name": "Sangiovese",
            "wines_count": 125094
        },
        {
            "name": "Canaiolo nero",
            "wines_count": 4879
        }
    ]
},
"vintages": [
    {
        "year": 2012,
        "price": 80,
        "statistics": {
            "ratings_count": 726,
            "ratings_average": 4.2
        },
        "image":
            "https://i.postimg.cc/hPHLGGHY/wine01.png",
        "reviews": [
            {
                "review_id": 3002,
                "user_id": {
                    "username": "bigpeacock714",
                    "thumbnail":
                        "https://randomuser.me/api/portraits/thumb/
                        women/91.jpg"
                }
            }
        ]
    }
]
```

```

    },
    "wine_id": {
      "id": 1,
      "name": "Grandi Annate Sangiovese",
      "year": 2012,
      "image":
        "https://i.postimg.cc/hPHLGGHY/wine01.png"
    },
    "rating": 5.0,
    "text": "Veramente un vino sorprendente!",
    "created_at": {
      "$date": "2021-08-11T08:53:24.000Z"
    },
    "likes_count": 13,
    "dislikes_count": 199
  }
],
"created_at": {
  "$date": "2014-04-18T21:06:31.839Z"
}
},
{
  "year": 2015,
  "price": 80,
  "statistics": {
    "ratings_count": 579,
    "ratings_average": 4.3
  },
  "image": "https://i.postimg.cc/hPHLGGHY/wine01.png",
  "reviews": [
    {
      "review_id": 3003,
      "user_id": {
        "username": "beautifulkoala533",
        "thumbnail":
          "https://randomuser.me/api/portraits/thumb/
            women/46.jpg"
      },
      "wine_id": {
        "id": 1,
        "name": "Grandi Annate Sangiovese",
        "year": 2015,
        "image":

```

```

        "https://i.postimg.cc/hPHLGGHY/wine01.png"
    },
    "rating": 4.0,
    "text": "Da provare.",
    "created_at": {
        "$date": "2024-03-11T21:40:29.000Z"
    },
    "likes_count": 13,
    "dislikes_count": 474
    }
],
"created_at": {
    "$date": "2018-03-15T01:47:19.197Z"
}
},
{
    "year": 2016,
    "price": 79,
    "statistics": {
        "ratings_count": 199,
        "ratings_average": 4.3
    },
    "image": "https://i.postimg.cc/hPHLGGHY/wine01.png",
    "reviews": [
        {
            "review_id": 3001,
            "user_id": {
                "username": "smalllion777",
                "thumbnail":
                    "https://randomuser.me/api/portraits/thumb/
                    men/75.jpg"
            },
            "wine_id": {
                "id": 1,
                "name": "Grandi Annate Sangiovese",
                "year": 2016,
                "image":
                    "https://i.postimg.cc/hPHLGGHY/wine01.png"
            },
            "rating": 4.3,
            "text": "Tannic, but bellissimooo",
            "created_at": {
                "$date": "2023-04-20T13:12:19.000Z"
            }
        }
    ]
}

```

```

        },
        "likes_count": 2,
        "dislikes_count": 167
    }
],
"created_at": {
    "$date": "2023-08-04T03:04:32.912Z"
}
}
],
"statistics": {
    "ratings_count": 4140,
    "ratings_average": 4.3
},
"created_at": {
    "$date": "2014-04-18T21:01:31.839Z"
}
}

```

Similarly to the approach used for the `users` collection, for each vintage we store the three most recent reviews. This list is updated in real time when a new review is submitted or an existing one is deleted: newly submitted reviews are immediately added to the top of the list, and deleted ones are instantly removed. However, the operation that ensures the array always contains the three most recent reviews in case of inconsistencies (e.g., after deletion) is scheduled to run once per day. This design choice avoids the need to access a separate collection when displaying the wine page, enabling faster retrieval of relevant reviews.

Wineries Collection

Here is an example of a document stored in this collection:

```

{
  "_id": {
    "$oid": "6888350fcb18a722a38c545b"
  },
  "name": "ARPEPE",
  "address": "Viale Europa, 30",
  "city": "Mantova",
  "zipcode": "46100",
  "province": "Mantova",
  "region": "Lombardia",
  "country": "Italia",
  "telephone": "+39 336 1157764",
  "email": "info@arpepe.com",

```

```

"website": "http://www.arpepe.com/",
"facebook": "https://www.facebook.com/arpepe",
"instagram": "https://www.instagram.com/arpepe/",
"images": [
  "https://i.postimg.cc/KYhkCGjh/winery092.jpg",
  "https://i.postimg.cc/CKbPVx4G/winery181.jpg",
  "https://i.postimg.cc/VvZzwRFb/winery007.jpg",
  "https://i.postimg.cc/gjVBqkxc/winery060.jpg",
  "https://i.postimg.cc/DZjzfnvZ/winery106.jpg",
  "https://i.postimg.cc/fRppNFQN/winery171.jpg",
  "https://i.postimg.cc/vmG8WcDT/winery113.jpg",
  "https://i.postimg.cc/9QgVqJ38/winery152.jpg"
],
"login": {
  "username": "arpepe3749",
  "password":
"$2b$12$ixjEU5Bo6Ioi8KGBj8SjtedJhJsg3sbMamSENzPLOqPoqCMFb7MGq"
},
"registered": {
  "date": {
    "$date": "2020-03-17T16:38:26.9322Z"
  },
  "age": 5
},
"picture": {
  "large": "https://i.postimg.cc/W3btL9tZ/winery112.jpg",
  "medium": "https://i.postimg.cc/VNv2xnm6/winery112.jpg",
  "thumbnail": "https://i.postimg.cc/LX6F6wtH/winery112.jpg"
}
}

```

Admins Collection

Here is an example of a document stored in this collection:

```

{
  "_id": {
    "$oid": "68363507cb18a722a38c545c"
  },
  "gender": "male",
  "name": {
    "title": "Mr.",
    "first": "Mario",
    "last": "Rossi"
  }
}

```

```
"email": "mario.rossi@example.com",
"telephone": "+39 392 3928397",
"login": {
  "username": "admin",
  "password":
"$2b$12$5M1i0bzaPCAusd3mFHvYtuFp6.6/G1Feo8Q5dgno2grW38XpKrSI2"
},
"registered": {
  "date": {
    "$date": "2020-03-24T14:30:15.123Z"
  },
  "age": 5
},
"dob": {
  "date": {
    "$date": "2000-03-24T14:30:15.123Z"
  },
  "age": 25
},
"picture": {
  "large": "https://randomuser.me/api/portraits/men/6.jpg",
  "medium": "https://randomuser.me/api/portraits/med/men/6.jpg",
  "thumbnail": "https://randomuser.me/api/portraits/thumb/men/6.jpg"
}
}
```

Regions Collection

Here is an example of a document stored in this collection:

```
{
  "_id": {
    "$oid": "6236350fcb18a722a38c545b"
  },
  "name": "Valle d'Aosta",
  "country": "Italia",
  "top_10_vintages_of_the_week": []
}
```

The field `top_10_vintages_of_the_week` is updated dynamically based on aggregation pipelines, which will be discussed in more detail later.

Countries Collection

Here is an example of a document stored in this collection:

```
{
```

```
    "_id": {
      "$oid": "6839350fcb18a722a38c545b"
    },
    "name": "Italia",
    "currency": {
      "code": "EUR",
      "name": "Euro",
      "prefix": "€"
    },
    "top_100_vintages_of_the_week": []
  }
```

The field `top_100_vintages_of_the_week` is updated dynamically based on aggregation pipelines, which will be discussed in more detail later.

Styles Collection

Here is an example of a document stored in this collection:

```
{
  "_id": {
    "$oid": "6836350fcb18a722a38c585b"
  },
  "name": "Vino Nobile di Montepulciano",
  "description": "You may not be familiar with Vino Nobile di Montepulciano, but you should be. Despite its long name, it makes delicious wines filled with preserved red cherry flavors. ...",
  "interesting_facts": [
    "Vino Nobile di Montepulciano was one of the original four DOCGs created in 1980.",
    "By law, the wines must be aged for 2 years to be called Vino Nobile and 3 years to qualify as Riserva.",
    "The area of Montepulciano is warmer than the rest of Tuscany.",
    "Vino Nobile di Montepulciano must contain at least 70% Prugnolo Gentile (aka Sangiovese)."
  ],
  "food": [
    {
      "name": "Manzo",
      "image": "https://i.postimg.cc/mDbRbCNG/manzo.jpg"
    },
    {
      "name": "Cacciagione (cervo, camoscio)",

```

```
        "image": "https://i.postimg.cc/Vshyh3Tw/cacciagione.jpg"
      }
    ],
    "grapes": [
      {
        "name": "Sangiovese",
        "wines_count": 125094
      },
      {
        "name": "Canaiolo nero",
        "wines_count": 4879
      }
    ],
    "taste": {
      "acidity": 4.0,
      "fizziness": null,
      "intensity": 3.5,
      "sweetness": 1.0,
      "tannin": 4.0
    }
  }
}
```

Counters Collection

Here is an example of a document stored in this collection:

```
{
  "_id": "reviews",
  "seq": 17466
},
{
  "_id": "wines",
  "seq": 4824
}
```

In the counters collection, we store the current sequential ID for managing the auto-incremented identifiers of the reviews and wines collections.

TopVintagesByOurQopType Collection

Here is an example of a document stored in this collection:

```
{
  "_id": {
    "$oid": "6846350fcb18a722a38c545b"
  },
}
```



```

    "type": "bianco",
    "vintages": [
      {
        "wine": "Lutum (Trebbiano d'Abruzzo)",
        "winery": "Tenuta Antonini",
        "year": 2023,
        "image": "https://i.postimg.cc/Sx82sf40/wine08.png",
        "ratings_count": 3,
        "quality": 5,
        "price": 14,
        "points": 100
      },
      {
        "wine": "Punggl Pinot Grigio",
        "winery": "Nals Margreid",
        "year": 2022,
        "image": "https://i.postimg.cc/fTyYwdkZ/wine04.png",
        "ratings_count": 6,
        "quality": 4.6,
        "price": 19.5,
        "points": 84.2
      },
      ...
    ]
  }

```

This collection stores the result of an **aggregation pipeline**, whose output is saved here for efficiency and fast access. It contains, for each wine **type**, a list of the **top 1000 vintages**, based on various metrics. Further details about the logic behind aggregations will be discussed in the **Analytics** section.

TopVintagesByQopType Collection

Here is an example of a document stored in this collection:

```

{
  "_id": {
    "$oid": "6836350fcb18a722a38c545b"
  },
  "type": "bianco",
  "vintages": [
    {
      "wine": "Chardonnay",
      "winery": "Barbanera",
      "year": 2023,

```

```

        "image": "https://i.postimg.cc/MHCZ2gvn/wine16.png",
        "ratings_count": 3,
        "quality": 4.166666666666667,
        "price": 6.4,
        "points": 100
    },
    {
        "wine": "Catarratto - Pinot Grigio",
        "winery": "Costalleggra",
        "year": 2022,
        "image": "https://i.postimg.cc/MKBKbZfV/wine18.png",
        "ratings_count": 1,
        "quality": 3.4,
        "price": 6.5,
        "points": 80.3
    },
    ...
]
}

```

This collection stores the result of an **aggregation pipeline**, whose output is saved here for efficiency and fast access. It contains, for each wine type, a list of the top 1000 vintages, based on various metrics. Further details about the logic behind aggregations will be discussed in the **Analytics** section.

TopVintagesRatingsType Collection

Here is an example of a document stored in this collection:

```

{
  "_id": {
    "$oid": "68363510cb18a722a38c5462"
  },
  "type": "rosato",
  "vintages": [
    {
      "wine": "Rosa dei Frati Riviera del Garda Classico Rosé",
      "winery": "Cà dei Frati",
      "year": 2023,
      "price": 12.4,
      "image": "https://i.postimg.cc/bYC2Fvg5/wine06.png",
      "ratings_average": 4.9,
      "ratings_count": 1
    },
    {

```

```
        "wine": "Furía di Calafuria",
        "winery": "Tormaresca",
        "year": 2022,
        "price": 24.7,
        "image": "https://i.postimg.cc/4491DM2Z/wine21.png",
        "ratings_average": 4.8,
        "ratings_count": 2
    },
    ...
]
}
```

This collection stores the result of an **aggregation pipeline**, whose output is saved here for efficiency and fast access. It contains, for each wine **type**, a list of the **top 1000 vintages**, based on various metrics. Further details about the logic behind aggregations will be discussed in the **Analytics** section.

TopWineriesRatings Collection

Here is an example of a document stored in this collection:

```
{
  "_id": {
    "$oid": "6836350fcb18a722a38c4cec"
  },
  "winery_username": "le_vigne_di_san_pietro7924",
  "thumbnail": "https://i.postimg.cc/26cRqGGn/winery105.jpg",
  "winery": "Le Vigne di San Pietro",
  "region": "Veneto",
  "country": "Italia",
  "ratings_average": 5,
  "ratings_count": 3
}
```

This collection contains the result of an **aggregation pipeline** that identifies the **top wineries**, based on multiple metrics. As with other similar structures, the purpose is to store precomputed results for quick access. The logic and methodology behind this aggregation will be detailed in the **Analytics** section.

TopWinesRatingsType Collection

Here is an example of a document stored in this collection:

```
{
  "_id": {
    "$oid": "68363510cb18a722a38c5469"
  },
  ...
}
```

```
"type": "rosato",
"wines": [
  {
    "wine": "Diamante Rosé",
    "winery": "Comincioli",
    "image": "https://i.postimg.cc/fTyYwdkZ/wine04.png",
    "prices_average": 20.98,
    "ratings_average": 4.675,
    "ratings_count": 4
  },
  {
    "wine": "Rosato",
    "winery": "Casa Grande della Quercia",
    "image": "https://i.postimg.cc/XJNpsMC3/wine09.png",
    "prices_average": 14.95,
    "ratings_average": 4.666666666666667,
    "ratings_count": 3
  },
  {
    "wine": "Morso Rosa Susumaniello Biologico",
    "winery": "Tenuta Viglione",
    "image": "https://i.postimg.cc/c4Pt05VM/wine07.png",
    "prices_average": 12.9,
    "ratings_average": 4.525,
    "ratings_count": 4
  },
  ...
]
}
```

This collection contains the result of an **aggregation pipeline** that identifies the **top wines**, based on multiple metrics. As with other similar structures, the purpose is to store precomputed results for quick access. The logic and methodology behind this aggregation will be detailed in the **Analytics** section.

3.3.3 Neo4j

Given the social nature of some features in our application, we adopted Neo4j to efficiently manage and explore user interactions. What follows is an overview of the graph's key elements and how they connect.

Justification of Neo4j Adoption

Neo4j was adopted to support the **social features** of our application, such as follow/unfollow operations, **generating suggestions** of accounts to follow, and building a **personalized feed** for each user. These features are inherently *graph-oriented*, making a graph database like Neo4j a natural and efficient choice. To avoid *redundant data storage* and maintain a clean separation of concerns between MongoDB and Neo4j, we chose to model user relationships **exclusively in Neo4j**. This design allows us to efficiently traverse connections and generate recommendations based on graph queries, without duplicating relationship data across both databases. Moreover, since our service does not require *real-time updates* of social metrics (e.g., number of followers or liked reviews), we embraced an *eventual consistency* approach: derived statistics and aggregated counters are updated in MongoDB periodically, while real-time relationship logic is handled in Neo4j. This trade-off enables us to prioritize **performance and scalability**, while still meeting the responsiveness required by our features.

3.3.3.1 Entities

User

The **User** node in Neo4j is a simplified version of the one stored in MongoDB. Since the graph database is used to support social features only, we decided to store just the essential information required for these functionalities: username, name, surname, and thumbnail.

Review

In our Neo4j graph, we chose to store only the latest three **Review** nodes for each user. Each review node contains only a subset of information, which is sufficient for generating personalized feeds and account suggestions: the review id, the wine id, the wine name, image, year and creation data, as well as the review rating and text. This design avoids unnecessary redundancy and ensures lightweight traversal during graph queries.

Winery

The **Winery** node represents a wine producer and is a simplified version of the entity stored in MongoDB. For the purposes of the social features implemented in Neo4j, we only store the username, name and thumbnail attributes.

3.3.3.2 Relationships

We defined two main relationship types: **FOLLOWS** and **WROTE**, each with specific domains and characteristics.

FOLLOWS

The **FOLLOWS** relationship is used to represent following connections between nodes

in the graph. Its primary domain is between users:

$$(u1:User) - [FOLLOWS] -> (u2:User)$$

This relationship is unidirectional: if a user *u1* follows another user *u2*, the reverse does not automatically hold. In the case of mutual following, two distinct **FOLLOWS** edges will exist. We opted for this design to ensure faster traversal performance in Neo4j. Traversing only outgoing relationships is generally more efficient than checking both incoming and outgoing ones along with an additional property filter. This choice supports faster execution of key queries.

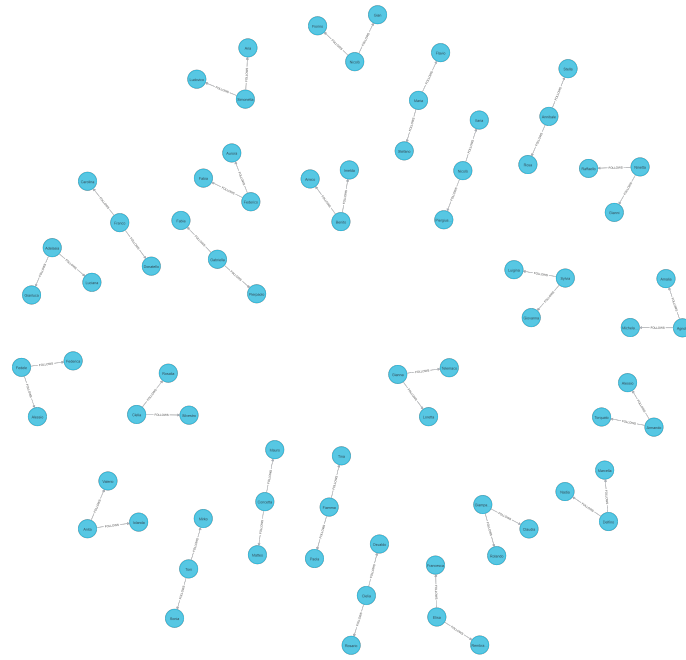


Figure 3.1: User nodes linked with **FOLLOWS** relationships.

Additionally, this relationship is also used between a user and a winery:

$$(u:User) - [FOLLOWS] -> (w:Winery)$$

This directionality is enforced by design: winery accounts cannot follow anyone, as they are intended only as followable entities. Wineries are included in the graph primarily to enable indirect recommendation strategies. For instance, users may receive winery suggestions based on which wineries are followed by people they follow.

WROTE

The **WROTE** relationship connects a **User** node to a **Review** node, representing the action of writing a review:

$$(u:User) - [WROTE] -> (r:Review)$$

This relationship allows us to retrieve all (the most recent) reviews authored by a

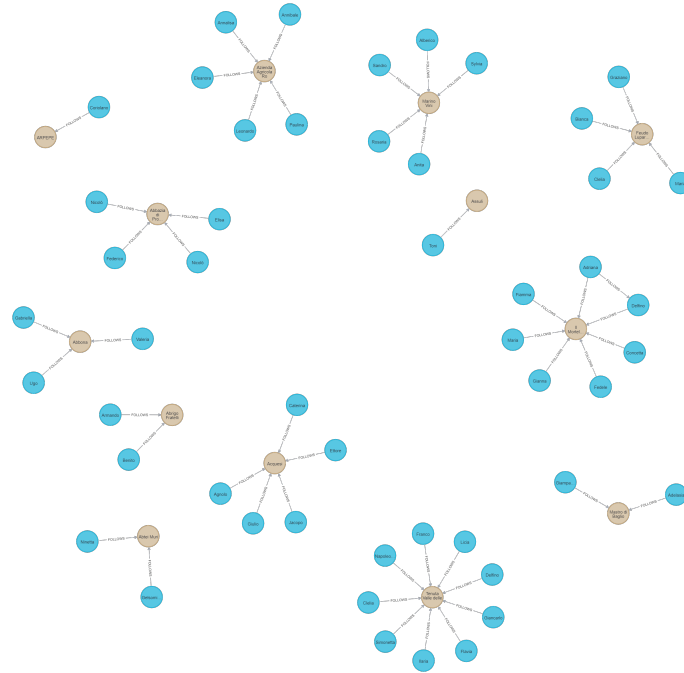


Figure 3.2: User and Winery nodes linked with FOLLOWS relationships.

given user, and is crucial in order to build the user's personal feed. Since each review is written by exactly one user, the **WROTE** relationship is single-directional and unique for each review node.

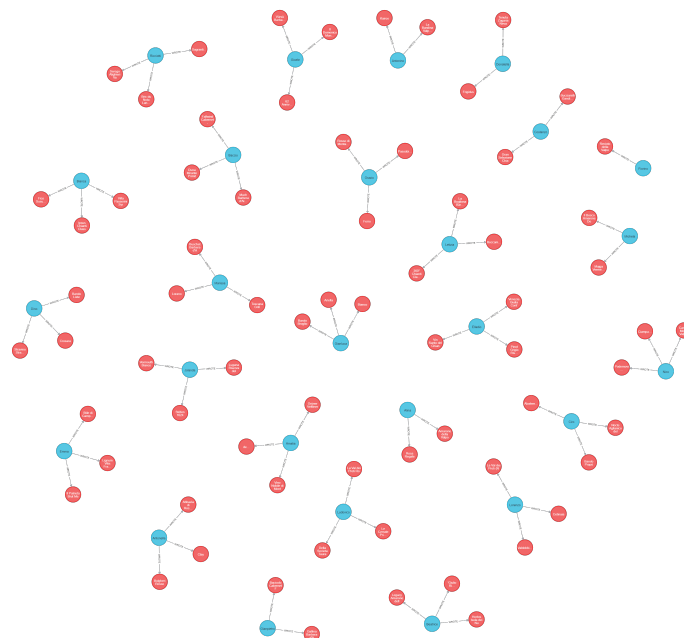


Figure 3.3: User and Review nodes linked with WROTE relationships.

Chapter 4

Implementation

4.1 Implementation Frameworks

For the development of our application, we chose to adopt the Spring Boot framework as the fundamental backbone. Spring Boot greatly simplifies the configuration and management of the entire application infrastructure, providing an embedded web server that allows us to rapidly build and deploy our backend services without the need for complex external setup. This feature alone has accelerated our development process by enabling a streamlined workflow and reducing boilerplate code. In addition, we leveraged the `@RestController` annotation extensively, which has proven invaluable in designing RESTful web services. This approach allowed us to map HTTP requests directly to Java methods in a clean and efficient manner, facilitating clear separation between the backend logic and the HTTP protocol details. The use of `@RestController` also helped us maintain consistency across the API endpoints, making the system easier to maintain and extend. Moreover, Spring Boot's powerful integration with Spring Security provided robust mechanisms to secure our application. This includes authentication, authorization, and session management, which were essential to safeguard sensitive user data and implement role-based access controls seamlessly. By using Spring Security within the Spring Boot ecosystem, we ensured a scalable and secure environment that meets the requirements of modern web applications.

4.2 Project Structure and Packages Organization

In our project, the code is arranged to promote modularity and simplify understanding. Different packages have been created, each dedicated to a distinct aspect of the application functionalities.

4.2.1 com.wineadvisor.wineadvisor.config

This package contains the configuration classes for the application, including security settings, database connections, and other environment-specific parameters.

- **SecurityConfig:** Configures the application security settings, defining authentication rules, route permissions, and stateless session management. It uses Spring Security with basic authentication and disables CSRF to simplify testing with Postman.
- **RepositoryConfig:** Sets up the `MongoTemplate` for interacting with MongoDB, disabling the automatic inclusion of the `_class` field in documents, thus keeping the stored data cleaner.

```
@Configuration
@EnableWebSecurity(securedEnabled = true, jsr256Enabled = true)
public class SecurityConfig {
    // =====
    // VARIABLES
    // =====
    private final AuthenticationService authenticationService;

    // =====
    // PUBLIC METHODS
    // =====

    // Authentication settings

    // Imposta il provider di autenticazione per l'autenticazione degli utenti
    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(authenticationService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    // Imposta il gestore di autenticazione per l'autenticazione degli utenti
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws Exception {
        return authConfig.getAuthenticationManager();
    }

    // Configura la catena di filtri di sicurezza per gestire le richieste HTTP
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf().disable() // Disabilita CSRF (utile per testare con Postman)
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // Nessuna sessione
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/authentication/login").anonymous() // Permetto l'accesso a /api/authentication/** solamente ad utenti non autenticati (endpoint di login)
                .requestMatchers(HttpMethod.POST, "/api/users").anonymous() // Permetto l'accesso a /api/users solamente ad utenti non autenticati per le richieste di tipo POST (endpoint di registrazione di un utente)
                .requestMatchers(HttpMethod.POST, "/api/wineries").anonymous() // Permetto l'accesso a /api/wineries solamente ad utenti non autenticati per le richieste di tipo POST (endpoint di registrazione di un'azienda vinicola)
                .requestMatchers("/api/admins/**").hasRole("ADMIN") // Permetto l'accesso a /api/admins solamente ad utenti con ruolo ADMIN (endpoint di gestione degli admin)
                .requestMatchers("/api/regions/**").hasRole("ADMIN") // Permetto l'accesso a /api/regions solamente ad utenti con ruolo ADMIN (endpoint di gestione delle regioni)
                .requestMatchers("/api/countries/**").hasRole("ADMIN") // Permetto l'accesso a /api/countries solamente ad utenti con ruolo ADMIN (endpoint di gestione delle nazioni)
                .anyRequest().authenticated() // Tutte le altre richieste richiedono autenticazione (senza specificare il ruolo)
            )
            .authenticationProvider(authenticationProvider())
            .httpBasic(httpBasic -> httpBasic.realmName("wineAdvisor")); // Configuro l'autenticazione Basic per Postman

        return http.build();
    }

    // END of auth. settings
}
```

Figure 4.1: SecurityConfig

4.2.2 com.wineadvisor.wineadvisor.controller

The controller package manages the HTTP requests.

- **AdminController:** Handles CRUD operations for administrator management, including CRUD operations and updates to username/password. Access is restricted to admins using security annotations.
- **AnalyticsController:** Handles CRUD operations to provide statistics about top wines, vintages, and wineries based on criteria like QOP, rating, and price. Also includes cleanup operations for monthly charts, accessible only by admins.

- **AuthenticationController**: Exposes an endpoint to authenticate users using Spring Security authentication manager. Returns a message indicating successful login along with the user's role.
- **CountryController**: Handles CRUD operations for countries. Allows paginated retrieval, renaming, and deletion of countries with appropriate validations.
- **RegionController**: Handles creation, retrieval, update, and deletion of regions. It supports filtering by country and pagination for fetching region data.
- **ReviewController**: Handles creation, retrieval, update, and deletion of reviews.
- **StyleController**: Handles creation, retrieval, update, and deletion of wine styles. Admins and wineries can create and manage styles through secured endpoints.
- **UserController**: Manages user-related operations such as account creation, profile updates, search, password changes, and interactions like likes, dislikes, and favorites. Secured with role-based access control.
- **WineController**: Manages wine-related endpoints, allowing wineries to create, update, and delete wines and vintages. Provides search and retrieval of wines with filtering and pagination, with role-based access control.
- **WineryController**: Handles winery management operations such as creating, updating, and deleting winery profiles. Supports search by winery name and includes security checks for authenticated updates and deletions.

```
@PostMapping("/{username}")
@Secured({"ROLE_ADMIN", "ROLE_USER"})
@PreAuthorize("#username == authentication.principal.username or hasRole('ROLE_ADMIN')")
public ResponseEntity<> updateUser(
    @NotBlank(message = "Username cannot be blank.") @PathVariable String username,
    @NotNull(message = "User update info cannot be null.") @Valid @RequestBody UpdateUserDTO updateUserDTO) {
    return ResponseEntity.status(HttpStatus.OK).body(userService.updateUser(username, updateUserDTO));
}
```

Figure 4.2: Snippet from **UserController**: endpoint to modify user profile information.

4.2.3 `com.wineadvisor.wineadvisor.model`

The model package represents the core domain entities of our application, encapsulating the key business data and related operations. It forms the foundation of our business logic by defining the main objects we work with. In this package, each class directly corresponds to a document stored within the MongoDB collections. This design choice ensures a straightforward mapping between the application's

data structures and the database schema, facilitating seamless data persistence and retrieval through Spring Boot's built-in support.

- **Review:** This class models the documents within the “reviews” collection. It contains the details of a user’s review on a specific wine, including the reviewer’s username, the wine reviewed, the review text, and the given rating.
- **Wine:** This class corresponds to the documents in the “wines” collection. It holds all relevant information about a wine, such as its name, vintage, style, associated winery, and other key attributes that define the product.
- **User:** This class represents documents from the “users” collection. It stores user-specific information like username, email, and preferences related to the WineAdvisor platform.
- **Winery:** This class models the documents within the “wineries” collection. It contains data about a winery, including its name, location, and a list of wines it produces.
- **Admin:** This class represents administrative users in the system. It contains details about users with elevated privileges who can manage platform settings and moderate content.
- **Country:** This class corresponds to the “countries” collection. It contains information about the countries related to wine production, including country name and possibly other geographic or regulatory data.
- **Region:** This class represents documents in the “regions” collection. It holds data about specific wine-producing regions within countries, often including associated wineries and local characteristics.
- **Style:** This class models the “styles” collection, capturing different wine styles or categories to classify wines by their typical flavor profiles or production methods.

Inside the `model` package, there is also a subpackage named `analytics`. This folder contains classes that correspond to collections storing the results of various aggregation queries used for analytics purposes. The classes within this subpackage include:

- `TopWinesRatingsType`
- `TopWineriesRatings`
- `TopVintagesRatingsType`
- `TopVintagesQopType`
- `TopVintagesOurQopType` (which extends `TopVintagesQopType`)

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "users")
@JsonInclude(JsonInclude.Include.NON_NULL)
public class User {
    @Id
    private ObjectId _id;

    private String gender;

    private Name name;

    private Address address;

    private String email;
    private String telephone;

    private Login login;

    private Registered registered;
    private Dob dob;

    private Picture picture;

    private ArrayList<ReviewEmbedded> reviews;
    private ArrayList<Long> likes;
    private ArrayList<Long> dislikes;

    @Field("wine_favorites")
    private ArrayList<WineFavorite> wineFavorites;

    @Field("wine_tips")
    private ArrayList<WineTip> wineTips;

    @Field("new_wine_tips")
    private ArrayList<WineTip> newWineTips;
```

Figure 4.3: User class: structure of the user model.

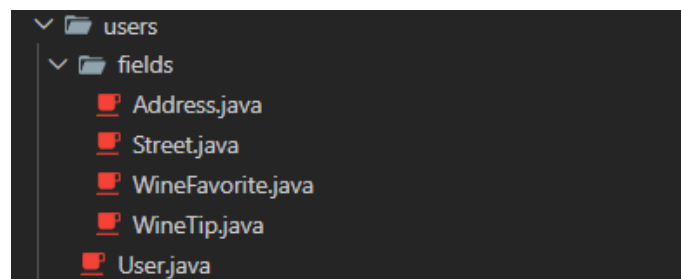


Figure 4.4: User class: folder structure.

4.2.4 com.wineadvisor.wineadvisor.service

This package implements the core business logic and service layers, processing data and applying application rules.

- AdminService
- AnalyticsService
- AuthenticationService
- CountryService

- RegionService
- IdCounterService - manages custom IDs for reviews and wines
- ReviewService
- StyleService
- UserService
- WineryService
- WineService

```

@Transactional
public User updateUser(String targetUsername, UpdateUserDTO updateUserDTO) throws ResourceNotFoundException, ResourceAlreadyExistsException {
    return userRepository
        .findByLoginUsername(targetUsername)
        .map(
            targetUser -> {
                boolean isThumbnailChanged = targetUser.getPicture().getThumbnail().equals(updateUserDTO.getPictureDTO().getThumbnail()) ? false : true;
                User userWithSameEmail = userRepository.findByEmail(updateUserDTO.getEmail()).orElse(null);
                if (userWithSameEmail != null && userWithSameEmail.getLogin().getUsername().equals(targetUser.getLogin().getUsername())) {
                    throw new ResourceAlreadyExistsException("User with username \"" + targetUser.getLogin().getUsername() + "\" not updatable because email \"" + updateUserDTO.getEmail() + "\" is already used by another user.");
                }
                targetUser = updateUserDTO.toUser(targetUser);
                if (isThumbnailChanged && targetUser.getReviews().size() > 0) {
                    for (ReviewInDb review : targetUser.getReviews()) {
                        review.getUserId().setThumbnail(updateUserDTO.getPictureDTO().getThumbnail());
                    }
                }
                // Aggiorno tutte le review dell'utente nella collection "reviews"
                updateReview_UserId_ThumbnailByUsername(targetUser.getLogin().getUsername(), targetUser.getPicture().getThumbnail());
            }
        )
        // Finalizzo gli aggiornamenti in modo da evitare inconsistenze nel database
        targetUser.adjustFieldsForUpdate();
        User savedUser = userRepository.save(targetUser);
        // Sincronizzazione con Redis
        userNeo4jRepository.updateUser(savedUser.getLogin().getUsername(), savedUser.getName().getFirst(), savedUser.getName().getLast(), savedUser.getPicture().getThumbnail());
        return savedUser;
    }
}
.orElseThrow(
    () -> new ResourceNotFoundException("User with username \"" + targetUsername + "\" not updatable because it does not exist.")
);
}

```

Figure 4.5: Snippet from UserService: service method for updating user information.

4.2.5 com.wineadvisor.wineadvisor.repository

The repository package serves as a dedicated layer for managing data access, isolating the underlying storage details from the rest of the application. It supports multiple databases, such as MongoDB and Neo4j, by offering tailored interfaces that leverage each database specific features. This structure promotes a clean separation of concerns and ensures efficient and consistent data operations across different storage technologies.

MongoDB Repositories

In our project, MongoDB repositories are interfaces that extend `MongoRepository`, allowing us to perform common database operations such as CRUD without writing boilerplate code. By annotating them with `@Repository`, Spring automatically generates the implementation at runtime, enabling seamless interaction with MongoDB collections through simple method definitions.

- AdminRepository
- CountryRepository
- RegionRepository
- ReviewRepository
- StyleRepository
- UserRepository
- WineRepository
- WineryRepository
- TopWinesRatingsTypeRepository
- TopWineriesRatingsRepository
- TopVintagesRatingsTypeRepository
- TopVintagesQopTypeRepository
- TopVintagesOurQopTypeRepository

```
@Repository
public interface UserRepository extends MongoRepository<User, ObjectId> {
    Optional<User> findByLogin_Username(String username);
    Optional<User> findByEmail(String email);
    Optional<User> findByReviews_ReviewId(Long id);

    Page<User> findByName_FirstContainingIgnoreCaseAndName_LastContainingIgnoreCase(String firstName, String lastName, Pageable pageable);
    Page<User> findByName_FirstContainingIgnoreCase(String name, Pageable pageable);
    Page<User> findByName_LastContainingIgnoreCase(String last_name, Pageable pageable);

    ArrayList<User> findByReviews_WineId_Id(Long wineId);
    ArrayList<User> findByReviews_WineId_IdAndReviews_WineId_Year(Long wineId, Integer vintageYear);
    ArrayList<User> findByLikesOrDislikes(Long id, long id2);
    ArrayList<User> findByAddress_Region(String region);
}
```

Figure 4.6: UserRepository

Neo4j Repositories

For Neo4j, the application uses repositories built with the Neo4jClient, allowing for fine-grained control over Cypher queries and data interactions. Instead of relying on auto-generated methods through Neo4jRepository, this approach enables the implementation of custom graph operations, optimized for specific logic such as limiting user reviews or managing dynamic relationships. This design ensures flexibility and full expressiveness in querying the graph database.

- ReviewNeo4jRepository
- UserNeo4jRepository
- WineryNeo4jRepository

```

@Repository
@RequiredArgsConstructor
public class UserNeo4jRepository {

    private final Neo4jClient neo4jClient;

    public void createUser(String username, String firstName, String lastName, String thumbnail) {
        Map<String, Object> params = new HashMap<>();
        params.put(key:"username", username);
        params.put(key:"firstName", firstName);
        params.put(key:"lastName", lastName);
        params.put(key:"thumbnail", thumbnail);

        neo4jClient.query("""
            CREATE (u:User {
                username: $username,
                firstName: $firstName,
                lastName: $lastName,
                thumbnail: $thumbnail
            })
            """)
            .bindAll(params)
            .run();
    }

    public Map<String, Object> findUserByUsername(String username) {
        return neo4jClient.query("""
            MATCH (u:User {username: $username})
            RETURN u.username AS username, u.firstName AS firstName, u.lastName AS lastName, u.thumbnail AS thumbnail
            """)
            .bind(username).to(name:"username")
            .fetch()
            .one()
            .orElse(other:null);
    }

    public void updateUser(String username, String firstName, String lastName, String thumbnail) {
        Map<String, Object> params = new HashMap<>();
        params.put(key:"username", username);
        params.put(key:"firstName", firstName);
        params.put(key:"lastName", lastName);
        params.put(key:"thumbnail", thumbnail);

        neo4jClient.query("""
            MATCH (u:User {username: $username})
            SET u.firstName = $firstName,
                u.lastName = $lastName,
                u.thumbnail = $thumbnail
            """)
            .bindAll(params)
            .run();
    }
}

```

Figure 4.7: Snippet from the UserNeo4jRepository

4.2.6 com.wineadvisor.wineadvisor.DTO

The DTO package contains the Data Transfer Objects used to encapsulate and exchanged data structure. Even though our project is backend-only, DTOs play a crucial role in decoupling the domain models from the exposed data, allowing for cleaner interfaces, improved maintainability, and better control over what is shared with the client. In particular, DTOs were used whenever the controller methods required a `@RequestBody` parameter, allowing for structured and validated input.

4.2.7 com.wineadvisor.wineadvisor.exception

This package contains all the custom exception classes and centralized error-handling logic used in the application. It allows us to provide meaningful and consistent error responses when unexpected conditions occur during backend processing. By defining custom exceptions (e.g., `BadRequestException`, `AccessDeniedException`) and mapping them to appropriate HTTP status codes, we ensured better clarity and control over API behavior. Additionally, the `GlobalExceptionHandler` class uses `@RestControllerAdvice` to intercept and handle both custom and standard

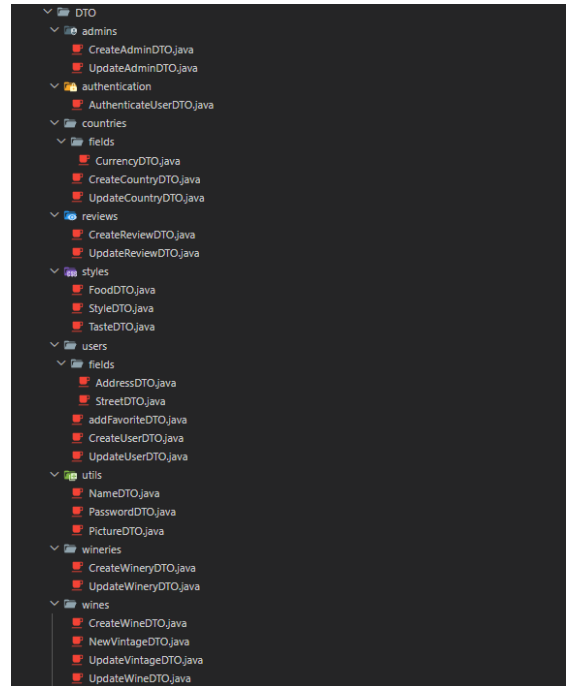


Figure 4.8: Overview of the DTO package organization.

```
@Data
@JsonPropertyOrder({ "gender", "name", "address", "email", "telephone", "date of birth", "profile picture" })
public class UpdateUserDTO {
    @Pattern(regexp = "^(male|female|other)$", message = "Gender must be one among \"male\", \"female\" and \"other\" (or blank).")
    @Schema(description = "Gender of the updated user", example = "male")
    private String gender;

    @NotNull(message = "Name info cannot be null.")
    @Valid
    @Schema(name = "name", description = "Name info of the updated user")
    @JsonProperty("name")
    private NameDTO nameDTO;

    @NotNull(message = "Address info cannot be null.")
    @Valid
    @Schema(name = "address", description = "Home address of the updated user")
    @JsonProperty("address")
    private AddressDTO addressDTO;

    @NotBlank(message = "Email cannot be blank.")
    @Email(message = "Email must be a valid email address.")
    @Schema(description = "Email of the updated user", example = "mariorossi@example.com")
    private String email;

    @Pattern(regexp = "^[\\+]?[0-9\\s\\-\\(\\)]+$", message = "Telephone must be a valid telephone number.")
    @Schema(description = "Telephone of the updated user", example = "+39 3331234567")
    private String telephone;

    @PastOrPresent(message = "Date must be in the past.")
    @Schema(name = "date of birth", description = "Date of birth of the updated user", example = "1970-01-01T00:00:00.000Z")
    @JsonProperty("date of birth")
    private Instant dob;

    @Schema(name = "profile picture", description = "Picture info of the updated user")
    @JsonProperty("profile picture")
    private PictureDTO pictureDTO;
}
```

Figure 4.9: UpdateUserDTO

exceptions in a unified way, improving debuggability and user feedback.

- **AccessDeniedException:** Thrown when a user attempts to access a resource or perform an action without the necessary permissions. It returns an HTTP 403 FORBIDDEN status.
- **BadRequestException:** Raised when a client request is invalid or malformed.

It helps return an HTTP 400 BAD REQUEST status.

- **DebugException:** A custom exception used for internal testing or debugging purposes. It always returns an HTTP 500 INTERNAL SERVER ERROR with a fixed message.
- **ResourceAlreadyExistsException:** Used when trying to create a resource that already exists in the system. It results in an HTTP 409 CONFLICT status.
- **ResourceNotFoundException:** Thrown when a requested resource cannot be found. This maps to an HTTP 404 NOT FOUND status.

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Figure 4.10: ResourceNotFoundException

- **GlobalExceptionHandler:** Centralizes exception handling for the entire application by mapping specific exceptions to meaningful HTTP responses. It ensures that client errors and server issues are returned with appropriate messages and status codes.

```
@Hidden
@RestControllerAdvice
public class GlobalExceptionHandler {

    ////////// AUTHENTICATION EXCEPTION //////////
    @ExceptionHandler({InternalAuthenticationServiceException.class})
    public ResponseEntity<> handleInternalAuthenticationServiceException(InternalAuthenticationServiceException e) {
        System.out.println(x:"--- WARN: InternalAuthenticationServiceException thrown and intercepted.");
        System.err.println("ERR: " + e.getMessage() + "\n\n");
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("ERR: " + e.getMessage());
    }

    @ExceptionHandler({BadCredentialsException.class})
    public ResponseEntity<> handleBadCredentialsException(BadCredentialsException e) {
        System.out.println(x:"--- WARN: BadCredentialsException thrown and intercepted.");
        System.err.println("ERR: " + e.getMessage() + "\n\n");
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("ERR: " + e.getMessage());
    }

    ////////// AUTHORIZATION EXCEPTION //////////
    @ExceptionHandler({AuthorizationDeniedException.class})
    public ResponseEntity<> handleAuthorizationDeniedException(AuthorizationDeniedException e) {
        System.out.println(x:"--- WARN: AuthorizationDeniedException thrown and intercepted.");
        System.err.println("ERR: " + e.getMessage() + ": user is trying to access to a resource which is not his.\n\n");
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body("ERR: " + e.getMessage() + ": the resource you are trying to access is not yours.");
    }

    @ExceptionHandler({AccessDeniedException.class})
    public ResponseEntity<> handleAccessDeniedException(AccessDeniedException e) {
        System.out.println(x:"--- WARN: AccessDeniedException thrown and intercepted.");
        System.err.println("ERR: ACCESS DENIED: user is trying to access to a resource which is not his.\n\n");
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body("ERR: ACCESS DENIED: the resource you are trying to access is not yours.");
    }
}
```

Figure 4.11: Snippet from the GlobalExceptionHandler: each exception is intercepted and mapped to a meaningful HTTP status and message.

4.3 MongoDB Operations

To provide a clearer and more organized structure, this section is divided into two parts: one dedicated to basic CRUD operations, and another focused on more complex queries such as aggregations and related analytics.

4.3.1 CRUD operations

CRUD operations have been implemented for each of the following collections: `users`, `reviews`, `wineries`, `wines`, `admins`, `countries`, `regions`, and `styles`. To avoid redundancy, we will not report all of them. Instead, we will use the `wines` collection as a representative example, providing code snapshots to illustrate the implementation.

Create a Wine

To create a new wine, the application performs the following steps, invoking the methods listed below:

- `public ResponseEntity<?> createWine(@RequestBody CreateWineDTO wine)`
- `public Wine addWine(CreateWineDTO createWineDTO, String wineryUsername)`
- `public Optional<Winery> findByLogin_Username(String username)`
- `public Optional<Country> findByName(String name)`
- `public Optional<Style> findByName(String name)`
- `public Wine save(Wine wine)`

The first method is an endpoint inside the `@RestController` and is responsible for handling the `POST` request when a winery user attempts to add a new wine. The method is secured with the `@Secured("ROLE_WINERY")` annotation to restrict access only to authenticated wineries. It extracts the username of the winery from the Spring Security context and delegates the creation logic to the service layer. In the service method `addWine()`, several validation and composition steps are carried out before persisting the new wine into the database:

- The winery is retrieved by its username using `wineryRepository.findByLogin_Username()`. If not found, a `ResourceNotFoundException` is thrown.
- Similarly, the winery's country is retrieved from the `countries` collection.
- A new `Wine` object is instantiated and populated with fields from the DTO and additional embedded documents such as winery, style, region, and country.
- If the specified wine style exists in the database, its information is embedded inside the wine; otherwise, the style is left `null`.

- The id of the wine is generated using a custom `IdCounterService`.
- Initial statistics and timestamps are set, and the final object is saved to the `wines` collection via `wineRepository.save()`.

The following figures show some of the most relevant code snippets involved in the wine creation process.

```
@PostMapping
@Secured("ROLE_WINERY")
public ResponseEntity<> createWine(
    @NotNull(message = "New wine info cannot be null.") @Valid @RequestBody CreateWineDTO wine) {
    String wineryUsername = (String) SecurityContextHolder.getContext().getAuthentication().getName();
    Wine savedWine = wineService.addWine(wine, wineryUsername);
    return ResponseEntity.status(HttpStatus.CREATED).header(headerName:"Location", "/api/wines/" + savedWine.getId()).body(savedWine);
}
```

Figure 4.12: `@RestController` method handling the wine creation request.

```
public Wine addWine(CreateWineDTO createWineDTO, String wineryUsername) throws ResourceNotFoundException {
    // Controllio che esista la winery
    Winery winery = wineryRepository.findByLogin_username(wineryUsername).orElseThrow(
        () -> new ResourceNotFoundException("Winery not found with username: " + wineryUsername + ".")
    );

    // Controllio che esista il country nella collection country
    Country country = countryRepository.findByName(winery.getCountry()).orElseThrow(
        () -> new ResourceNotFoundException("Country not found with name: " + winery.getCountry() + ".")
    );

    Wine wine = new Wine();
    wine.setId(idCounterService.generateSequence(seqName:"wines"));
    wine.setName(createWineDTO.getName());
    wine.setType(createWineDTO.getType());
    wine.setIsNatural(createWineDTO.getIsNatural());

    wine.setTaste(taste:null);

    Optional<Style> style_to_find = styleRepository.findByName(createWineDTO.getStyle());
    if(style_to_find.isEmpty()) {
        wine.setStyle(style:null);
    } else {
        Style style = style_to_find.get();

        wine.setStyle(new StyleEmbedded());
        wine.getStyle().setName(style.getName());
        wine.getStyle().setDescription(style.getDescription());
        wine.getStyle().setInterestingFacts(style.getInterestingFacts());
        wine.getStyle().setFood(style.getFood());
        wine.getStyle().setGrapes(style.getGrapes());
    }

    wine.setVintages(new ArrayList<Vintage>());
    wine.setStatistics(new Statistics());
    wine.getStatistics().setRatingsAverage(ratingsAverage:0.0);
    wine.getStatistics().setRatingsCount((long) 0);

    wine.setWinery(new WineryEmbedded());
    wine.getWinery().setName(winery.getName());
    wine.getWinery().setUsername(winery.getLogin().getUsername());
    wine.getWinery().setThumbnail(winery.getPicture().getThumbnail());

    wine.setRegion(new RegionEmbedded());
    wine.getRegion().setName(winery.getRegion());
    wine.getRegion().setCountry(new CountryEmbedded());
    wine.getRegion().getCountry().setName(country.getName());
    wine.getRegion().getCountry().setCurrency(country.getCurrency());

    wine.setCreatedAt(Instant.now());

    return wineRepository.save(wine);
}
```

Figure 4.13: Service method that creates and saves a new wine.

Search for Wines

To search for wines based on multiple filters, the application performs the following steps, invoking the methods listed below:

- `public ResponseEntity<?> searchWines(...)`
- `public Page<Wine> searchWines(...)`
- `public Page<Wine> findByNameContainingIgnoreCaseAndWinery
_UsernameContainingIgnoreCaseAndRegion_NameContainingIgnoreCaseAndRegion
_Country_NameContainingIgnoreCaseAndTypeContainingIgnoreCaseAndStyle
_Grapes_NameContainingIgnoreCaseAndStatistics
_RatingsAverageGreaterThanOrEqualToAndVintages_PriceBetween(...)`

The first method is a GET endpoint inside the `@RestController` that handles search requests from the client. It accepts various optional query parameters such as wine name, winery username, region, country, type, grape, price range, and minimum average rating. If not provided, the parameters are initialized with default values in order to construct a valid query. Basic validation is performed at this level: for example, prices and ratings must be non-negative, and `minPrice` cannot exceed `maxPrice`. If the validation fails, a `BadRequestException` is thrown. The controller delegates the logic to the service method `searchWines()`, which constructs a paginated query on the `wines` collection. The method uses a derived query from the `WineRepository` to apply all filters simultaneously:

- `name` is matched using a case-insensitive substring on `name`.
- `winery` filters the embedded field `winery.username`.
- `region` and `country` match `region.name` and `region.country.name`, respectively.
- `type` is matched as a case-insensitive substring.
- `grape` is matched against `style.grapes.name`.
- `minPrice` and `maxPrice` define a price interval applied on `vintages.price`.
- `minAverageRating` sets a lower bound on `statistics.ratingsAverage`.

The service applies pagination using a `PageRequest` with fixed page size. After executing the query, the result is validated through the `checkReturnedPage()` utility method:

- If no matching wines are found, a `ResourceNotFoundException` is thrown.
- If the requested page index is too high for the result set, a `BadRequestException` is thrown.

If the query is successful, the method returns a `Page<Wine>` object to the controller, which wraps it in a 200 OK `ResponseEntity`. The following figures show some of the most relevant code snippets involved in the wine search process.

```

@PostMapping("/search")
public ResponseEntity<> searchWines(
    @RequestParam(required = false, name = "page number", defaultValue = "0")
    @PositiveOrZero Integer page,
    @RequestParam(required = false)
    @Schema(description = "Wine name", example = "Il Pettirosso")
    String name,
    @RequestParam(required = false)
    @Schema(description = "Winery username", example = "arpepe1749")
    String winery,
    @RequestParam(required = false)
    @Schema(description = "Region name", example = "Lombardia")
    String region,
    @RequestParam(required = false)
    @Schema(description = "Country name", example = "Italia")
    String country,
    @RequestParam(required = false)
    @Pattern(regexp = "rosso|bianco|rosato|spumante|vino macerato|vino da dessert|vino liquoroso|vino aromatizzato", message = "Type must be one of: rosso, bianco, rosato, spumante, vino macerato, vino da dessert, vino liquoroso, vino aromatizzato.")
    @Schema(description = "Wine type info", example = "rosso")
    String type,
    @RequestParam(required = false)
    @Schema(description = "Grape name", example = "Nebbiolo")
    String grape,
    @RequestParam(required = false)
    @DecimalMin(value = "0.0", inclusive = true, message = "Price must be at least 0.")
    @Schema(description = "Min vintage price", example = "70.0")
    Double minPrice,
    @RequestParam(required = false)
    @DecimalMin(value = "0.0", inclusive = true, message = "Price must be at least 0.")
    @Schema(description = "Max vintage price", example = "120.0")
    Double maxPrice,
    @RequestParam(required = false)
    @DecimalMin(value = "0.0", inclusive = true, message = "Rating must be at least 0.")
    @DecimalMax(value = "5.0", inclusive = true, message = "Rating must be at most 5.")
    @Schema(description = "Min average wine rating", example = "2.0")
    Double minAverageRating
) throws BadRequestException {
    name = (name == null) ? "" : name;
    winery = (winery == null) ? "" : winery;
    region = (region == null) ? "" : region;
    country = (country == null) ? "" : country;
    type = (type == null) ? "" : type;
    grape = (grape == null) ? "" : grape;
    minPrice = (minPrice == null) ? 0.0 : minPrice;
    maxPrice = (maxPrice == null) ? 2000.0 : maxPrice;
    minAverageRating = (minAverageRating == null) ? 0.0 : minAverageRating;

    if (minPrice > maxPrice) {
        throw new BadRequestException(message: "Min price cannot be greater than max price.");
    }

    return ResponseEntity.status(HttpStatus.OK).body(wineService.searchWines(page, name, winery, region, country, type, grape, minPrice, maxPrice, minAverageRating));
}

```

Figure 4.14: `@RestController` method handling the wines search request.

```

public Page<Wine> searchWines(
    Integer page,
    String name,
    String winery,
    String region,
    String country,
    String type,
    String grape,
    Double min,
    Double max,
    Double minAverageRating) {
    Page<Wine> wines = wineRepository.findByNameContainingIgnoreCaseOrWineryUsernameContainingIgnoreCaseOrRegion_NameContainingIgnoreCaseOrCountry_NameContainingIgnoreCaseOrTypeContainingIgnoreCaseOrGrape_NameContainingIgnoreCaseOrStatistics_RatingAverageGreaterThanOrEqualTo_Vintage_PriceBetween_
    PageRequest.of(page, PageSize, name, winery, region, country, type, grape, minAverageRating, min, max);
    checkReturnedPageWines, notFoundMessage: "No wines found with the specified filters.");
    return wines;
}

```

Figure 4.15: Service method that searches for wines.

```

private void checkReturnedPage(Page<Wine> page, String notFoundMessage) throws ResourceNotFoundException, BadRequestException {
    if (page.getTotalElements() == 0) {
        throw new ResourceNotFoundException(notFoundMessage);
    }
    if (page.getPageable().getPageNumber() >= page.getTotalPages()) {
        throw new BadRequestException(message: "Page requested too high.");
    }
}

```

Figure 4.16: Method used to validate the results of a paginated wine search, ensuring that the requested page exists and contains data.

Update a Wine

To update the details of an existing wine, the application performs the following steps, invoking the methods listed below:

- `public ResponseEntity<?> updateWine(@RequestBody @Valid UpdateWineDTO wine)`

- `public Wine updateWine(UpdateWineDTO updatedWine, String username)`
- `public Optional<Wine> findById(String id)`
- `public Optional<Wine> findByIdAndWinery_Username(String id, String username)`
- `public Optional<Style> findByName(String name)`
- `public ArrayList<User> findByReviews_WineId_Id(String id)`
- `public ArrayList<Review> findByWineId_Id(String id)`
- `public Wine save(Wine wine)`

The first method is a PUT endpoint defined in the `@RestController`, secured with the `@Secured("ROLE_WINERY")` annotation to allow access only to authenticated wineries. It extracts the username from the Spring Security context and delegates the update operation to the service layer. In the service method `updateWine()`, the following steps are performed:

- The target wine is retrieved by its ID. If not found, a `ResourceNotFoundException` is thrown.
- A secondary check verifies that the wine belongs to the winery attempting the update by querying `findByIdAndWinery_Username()`. If the match fails, a `ResourceNotFoundException` is raised.
- The wine's main fields such as `name`, `type`, and `isNatural` are updated.
- If a valid style is found in the database, it is embedded into the wine object; otherwise, the style field is set to `null`.
- The updated wine name is propagated to all `ReviewEmbedded` objects in the `users` collection and to all reviews in the `reviews` collection to ensure data consistency.
- After all updates are applied, the wine is saved back into the `wines` collection using `wineRepository.save()`.

The update operation is marked with `@Transactional` to ensure atomicity. If any part of the process fails, the transaction is rolled back. The following figures show some of the most relevant code snippets involved in the wine update process.

```

@PutMapping
@Secured({ "ROLE_WINERY" })
public ResponseEntity<?> updateWine(
    @RequestBody @Valid UpdateWineDTO wine) {
    // Prendo username della winery che vuole modificare il vino
    String username = SecurityContextHolder.getContext().getAuthentication().getName();
    Wine updatedWine = wineService.updateWine(wine, username);
    return ResponseEntity.status(HttpStatus.OK).body(updatedWine);
}

```

Figure 4.17: @RestController method handling the wine update request.

```

@Transactional
public Wine updateWine(UpdateWineDTO updatedWine, String username) throws ResourceNotFoundException {
    return wineRepository.findById(updatedWine.getId())
        .map(wine -> {
            // Controllo che il vino sia della winery username
            if(wineRepository.findByIdAndWinery_Username(updatedWine.getId(), username).isEmpty()){
                throw new ResourceNotFoundException("Winery " + username + " does not own wine with id " + updatedWine.getId() + ".");
            }

            wine.setName(updatedWine.getName());
            wine.setType(updatedWine.getType());
            wine.setIsNatural(updatedWine.getIsNatural());

            Optional<Style> style_to_find = styleRepository.findByName(updatedWine.getStyle());
            if(style_to_find.isEmpty()) {
                wine.setStyle(style:null);
            } else {
                Style style = style_to_find.get();

                wine.setStyle(new StyleEmbedded());
                wine.getStyle().setName(style.getName());
                wine.getStyle().setDescription(style.getDescription());
                wine.getStyle().setInterestingFacts(style.getInterestingFacts());
                wine.getStyle().setFood(style.getFood());
                wine.getStyle().setGrapes(style.getGrapes());
            }

            // Devo aggiornare il nome anche nelle reviews embedded nella collection users
            ArrayList<User> users = userRepository.findByReviews_WineId_Id(updatedWine.getId());
            if (!users.isEmpty()){
                for (User user : users){
                    for (ReviewEmbedded review : user.getReviews()){
                        if (review.getId().equals(updatedWine.getId())){
                            review.getId().setName(updatedWine.getName());
                        }
                    }
                    userRepository.save(user);
                }
            }

            // Devo aggiornare il nome anche nelle reviews della collection reviews
            ArrayList<Review> reviews = reviewRepository.findByWineId_Id(updatedWine.getId());
            if (!reviews.isEmpty()){
                for (Review review : reviews){
                    review.getId().setName(updatedWine.getName());
                    reviewRepository.save(review);
                }
            }

            return wineRepository.save(wine);
        })
        .orElseThrow(
            () -> new ResourceNotFoundException("Wine with id " + updatedWine.getId() + " not found.")
        );
}

```

Figure 4.18: Service method that updates a wine.

Delete a Wine

To delete a wine, the following steps are performed by the application, invoking the methods listed below:

- `public ResponseEntity<?> deleteWine(@PathVariable Long id)`
- `public void deleteWineById(Long wineId, String username)`
- `public Optional<Wine> findById(Long id)`
- `public Optional<Wine> findByIdAndWinery_Username(Long id, String username)`
- `public ArrayList<Review> findByWineId_Id(Long id)`
- `public Iterable<User> findAll()`
- `public void deleteAllByWineId_Id(Long id)`
- `public void deleteById(Long id)`

The process starts from a `DELETE` endpoint accessible only to users with roles `ROLE_ADMIN` or `ROLE_WINERY`, secured using the `@Secured` annotation. The ID of the wine to be deleted is extracted from the URL path, and the authenticated user's username is obtained via the Spring Security context. The deletion is then delegated to the service layer. In the `deleteWineById()` service method, the application performs the following:

- Checks whether the wine with the given ID exists in the database. If not, a `ResourceNotFoundException` is thrown.
- Verifies that the wine is owned by the winery making the request. If not, another `ResourceNotFoundException` is raised.
- Retrieves all reviews associated with the wine using `findByWineId_Id()`, and removes each review:
 - From the embedded list of reviews in each `User` document.
 - From the lists of liked and disliked reviews of users, by checking if the review ID matches.
 - If a user was modified, they are saved back to the database using `userRepository.save()`.
- Deletes all reviews of the wine from the `reviews` collection via `deleteAllByWineId_Id()`.
- Finally, the wine is removed from the `wines` collection using `deleteById()`.

This method is annotated with `@Transactional` to ensure that all deletions and updates are performed atomically. If any part of the process fails, the transaction is rolled back, preserving data consistency. The code snippets below show the core components of the wine deletion functionality.

```

@DeleteMapping("/{id}")
@Secured({ "ROLE_ADMIN", "ROLE_WINERY" })
public ResponseEntity<> deleteWine(
    @PathVariable @NotNull(message = "ID cannot be null.") @Positive(message = "ID cannot be negative.") Long id) {
    // Prendo username della winery che vuole aggiungere la vintage al vino
    String username = SecurityContextHolder.getContext().getAuthentication().getName();
    wineService.deleteWineById(id, username);
    return ResponseEntity.status(HttpStatus.OK).body(body:"Wine deleted successfully.");
}

```

Figure 4.19: @RestController method handling the wine deletion request.

```

@Transactional
public void deleteWineById(Long wineId, String username) throws ResourceNotFoundException {
    // Controllo che il vino specificato esista
    if (wineRepository.findById(wineId).isEmpty()){
        throw new ResourceNotFoundException("Wine with id " + wineId + " not found.");
    }

    // Controllo che il vino sia della winery username
    if(wineRepository.findByIdAndWinery_Username(wineId, username).isEmpty()){
        throw new ResourceNotFoundException("Winery " + username + " does not own wine with id " + wineId + ".");
    }

    // Se elimino un vino, devo eliminare anche tutte le recensioni fatte su quel vino
    // e togliere l'id di quelle recensioni dagli array "likes"/"dislikes" degli users.
    ArrayList<Review> reviews_to_delete = reviewRepository.findByWineId_Id(wineId);
    ArrayList<User> users = (ArrayList<User>) userRepository.findAll();
    for (User user : users){
        boolean modified = user.getReviews().removeIf(r -> r.getWineId().getId().equals(wineId));
        if (user.getLikes().removeIf(likeId -> reviews_to_delete.stream().anyMatch(r -> r.getId().equals(likeId)))) {
            modified = true;
        }
        if (user.getDislikes().removeIf(dislikeId -> reviews_to_delete.stream().anyMatch(r -> r.getId().equals(dislikeId)))) {
            modified = true;
        }
        if (modified) {
            userRepository.save(user);
        }
    }

    reviewRepository.deleteAllByWineId_Id(wineId);
    wineRepository.deleteById(wineId);
}

```

Figure 4.20: Service method that deletes a wine.

4.3.2 MongoDB Aggregations for Analytics

Before exploring the details of the aggregation pipelines implemented in the system, it is important to clarify how analytics are handled within the application. Rather than computing aggregations on demand, the system periodically executes predefined aggregation pipelines and stores their results in dedicated collections within the MongoDB database. This design choice improves performance and scalability, allowing clients to simply retrieve precomputed analytics through specific GET endpoints. Each aggregation is therefore associated with a corresponding collection that is updated at regular intervals, ensuring that the data remains relevant and up-to-date without introducing overhead during query execution.

Daily Update of Vintage Statistics

This aggregation pipeline is scheduled to run daily at midnight. It updates the `statistics` field for each `vintage` within the `wines` collection. The statistics in-

clude the number of ratings (`ratings_count`) and the average rating (`ratings_average`) computed from the `reviews` collection. The pipeline performs the following steps:

1. `$unwind` the `vintages` array to process each vintage individually.
2. `$lookup` reviews that match both the wine's ID and the vintage year.
3. `$project` a new field `statistics` containing the computed values.

To ensure the performance of this pipeline, a compound index on the fields `wine_id.id` and `wine_id.year` is created in the `reviews` collection. The aggregation results are then used to update the corresponding vintage `statistics` field in the `wines` collection via individual update operations.

Daily Update of Wine Statistics

This aggregation pipeline also runs daily at midnight and is responsible for updating the overall `statistics` field of each wine (not per vintage). It calculates the total number of ratings and their average, based on all the reviews referring to that wine. The steps involved are:

1. `$lookup` reviews for each wine using the wine's ID.
2. `$project` the new `statistics` field for each wine.
3. `$merge` the updated statistics back into the `wines` collection, overwriting existing values when matched.

A single-field index on `wine_id.id` is ensured for the `reviews` collection to support efficient matching. Unlike the vintage update, this pipeline uses the `$merge` operator to apply bulk updates directly within the aggregation framework.

Daily Update of Wines Count per Grape in Styles and Wines Collections

This aggregation pipeline is scheduled to run daily at midnight and updates the `wines_count` field associated with each grape variety both within the nested `style.grapes` array in the `wines` collection and in the `grapes` array of the `styles` collection. The process consists of:

1. `$unwind` the `style.grapes` array in the `wines` collection to handle each grape variety separately.
2. `$group` by the grape name to count how many wines contain that grape (`wines_count`).

The aggregation results are iterated to:

- Update all matching grape entries in `style.grapes` of the `wines` collection by setting the new `wines_count`.

```

@Transactional
@Scheduled(cron = "0 0 0 * * ?") // Ogni giorno a mezzanotte
protected void updateStatisticsPerVintage() {
    System.out.println(x:"--- INFO: Declaring aggregation pipeline stages for vintage statistics update.");
    List<Document> pipeline = Arrays.asList(
        new Document(key:"$unwind", value:"$vintages"),
        new Document(key:"$lookup",
            new Document(key:"from", value:"reviews")
                .append(key:"let",
                    new Document(key:"wineId", value:"$_id")
                        .append(key:"vintageYear", value:"$vintages.year"))
                .append(key:"pipeline", Arrays.asList(new Document(key:"$match",
                    new Document(key:"$expr",
                        new Document(key:"$and", Arrays.asList(
                            new Document(key:"$eq", Arrays.asList(...a:"$wine_id.id", "$$wineId")),
                            new Document(key:"$eq",
                                Arrays.asList(...a:"$wine_id.year", "$$vintageYear"))))))),
                    new Document(key:"$project",
                        new Document(key:"rating", value:1))))
                .append(key:"as", value:"matched_reviews")),
        new Document(key:"$project",
            new Document(key:"_id", value:1)
                .append(key:"year", value:"$vintages.year")
                .append(key:"statistics",
                    new Document(key:"ratings_count",
                        new Document(key:"$size", value:"$matched_reviews")
                            .append(key:"ratings_average",
                                new Document(key:"$cond", Arrays.asList(
                                    new Document(key:"$gt",
                                        Arrays.asList(new Document(key:"$size",
                                            value:"$matched_reviews"), 0L)),
                                    new Document(key:"$avg", value:"$matched_reviews.rating"),
                                    new BsonNull())))))));
        // Assicuro che esistano gli indici FUNZIONALI alla pipeline.
    mongoTemplate
        .indexOps(collectionName:"reviews")
        .ensureIndex(
            new Index()
                .on(key:"wine_id.id", Sort.Direction.ASC)
                .on(key:"wine_id.year", Sort.Direction.ASC)
                .named(name:"index_for_vintage_statistics_update")
        );
    System.out.println(x:"--- INFO: Ensured index on fields \"wine_id.id\" and \"wine_id.year\" for collection \"reviews\".");
    System.out.println(x:"--- INFO: Executing aggregation pipeline for vintage statistics update...");
    AggregateIterable<Document> results = mongoTemplate.getCollection(collectionName:"wines").aggregate(pipeline).allowDiskUse(allowDiskUse:true);
    System.out.println(x:"--- INFO: Aggregation pipeline executed successfully.");
    System.out.println(x:"--- INFO: Processing results from aggregation pipeline...");
    for (Document doc : results) {
        Object idObj = doc.get(key:"_id");
        Object yearObj = doc.get(key:"year");
        Document stats = (Document) doc.get(key:"statistics");
        // Verifico che tutti i campi siano presenti
        if (idObj == null || yearObj == null || stats == null) {
            continue;
        }
        Long wineId = ((Number) idObj).longValue();
        int year = ((Number) yearObj).intValue();
        // Query per selezionare il vino e la vintage giusta
        Query query = new Query(Criteria.where(key:"_id").is(wineId)
            .and(key:"vintages.year").is(year));
        // Update per sovrascrivere il campo statistics nella vintage corretta
        Update update = new Update().set(key:"vintages.$statistics", stats);
        mongoTemplate.updateFirst(query, update, collectionName:"wines");
    }
    System.out.println(x:"--- INFO: Vintage statistics updated successfully.\n\n");
}

```

Figure 4.21: updateStatisticsPerVintage()

- Update the **grapes** array inside the **styles** collection analogously.

The updates leverage **arrayFilters** to target the correct grape entries within arrays, and **updateMulti** ensures multiple documents are updated where applicable. This approach guarantees that the grape wine counts remain consistent and up-to-date across related collections, facilitating accurate analytics and reporting.

Weekly Update of Top 10 Vintages per Region

This aggregation pipeline is scheduled to execute every Monday at 2:00 AM. It computes the top 10 most reviewed wine vintages for each region and updates the

```

@Scheduled(cron = "0 0 0 * * ?") // Ogni giorno a mezzanotte
protected void updateStatisticsPerWine() {
    System.out.println(x:"--- INFO: Declaring aggregation pipeline stages for wine statistics update.");
    List<Document> pipeline = Arrays.asList(
        new Document(key:"$project",
            new Document(key:"_id", value:1L)),
        new Document(key:"$lookup",
            new Document(key:"from", value:"reviews")
                .append(key:"localField", value:"_id")
                .append(key:"foreignField", value:"wine_id.id")
                .append(key:"as", value:"matched_reviews")),
        new Document(key:"$project",
            new Document(key:"_id", value:1L)
                .append(key:"statistics",
                    new Document(key:"ratings_count",
                        new Document(key:"$size", value:"$matched_reviews")
                            .append(key:"ratings_average",
                                new Document(key:"$cond",
                                    Arrays.asList(
                                        new Document(key:"$gt",
                                            Arrays.asList(
                                                new Document(key:"$size",
                                                    value:"$matched_reviews"), 0L)),
                                        new Document(key:"$avg", value:"$matched_reviews.rating"),
                                        new BsonNull())))),
                    new Document(key:"$merge",
                        new Document(key:"into", value:"wines")
                            .append(key:"on", value:"_id")
                            .append(key:"whenMatched", Arrays.asList(new Document(key:"$set",
                                new Document(key:"statistics", value:"$$new.statistics"))))
                            .append(key:"whenNotMatched", value:"discard"))));

    // Assicuro che esistano gli indici FUNZIONALI alla pipeline.
    mongoTemplate
        .indexOps(collectionName:"reviews")
        .ensureIndex(
            new Index()
                .on(key:"wine_id.id", Sort.Direction.ASC)
                .named(name:"index_for_wine_statistics_update")
        );
    System.out.println(x:"--- INFO: Ensured index on field \"wine_id.id\" for collection \"reviews\".");

    System.out.println(x:"--- INFO: Executing aggregation pipeline for wine statistics update...");
    mongoTemplate.getCollection(collectionName:"wines").aggregate(pipeline).allowDiskUse(allowDiskUse:true).toCollection();
    System.out.println(x:"--- INFO: Wine statistics updated successfully.\n\n");
}

```

Figure 4.22: updateStatisticsPerWine()

regions collection accordingly. A vintage is defined as a unique pair of wine and year. The pipeline operates on the `reviews` collection and proceeds through the following stages:

1. `$lookup` joins the `reviews` collection with the `wines` collection by matching the `id` field, enriching each review with the wine region.
2. `$unwind` flattens the joined results.
3. `$project` extracts only the relevant fields: wine information (ID, name, image, year) and the region.
4. `$group` clusters the data by wine ID and vintage year, computing the number of reviews for each combination.
5. `$sort` ranks the combinations by review count in descending order.

```

@Transactional
@Scheduled(cron = "0 0 0 * * ?") // Ogni giorno a mezzanotte
protected void updateWinesCountInGrapesPerWine() {
    System.out.println(x:"--- INFO: Declaring aggregation pipeline stages for grapes count update.");
    List<Document> pipeline = Arrays.asList(
        new Document(key:"$unwind",
            new Document(key:"path", value:"$style.grapes")),
        new Document(key:"$group",
            new Document(key:"_id", value:"$style.grapes.name")
                .append(key:"wines_count",
                    new Document(key:"$sum", value:1)))),
        new Document(key:"$sum", value:1));

    System.out.println(x:"--- INFO: Executing aggregation pipeline for grapes count update...");
    AggregateIterable<Document> results = mongoTemplate.getCollection(collectionName:"wines").aggregate(pipeline).allowDiskUse(allowDiskUse:true);
    System.out.println(x:"--- INFO: Aggregation pipeline executed successfully.");

    System.out.println(x:"--- INFO: Processing results from aggregation pipeline...");
    for (Document doc : results) {
        String grapeName = doc.getString(key:"_id");
        Long winesCount = doc.getLong(key:"wines_count");

        if (grapeName == null || winesCount == null) {
            continue;
        }

        // Seleziona tutti i vini che contengono questa grape
        Query query_on_wines = new Query(Criteria.where(key:"style.grapes.name").is(grapeName));
        Query query_on_styles = new Query(Criteria.where(key:"grapes.name").is(grapeName));

        // Aggiorna ogni entry dentro style.grapes con quel nome
        Update update_on_wines = new Update().set(key:"style.grapes.$[g].wines_count", winesCount);
        Update update_on_styles = new Update().set(key:"grapes.$[g].wines_count", winesCount);

        // Definisce il filtro per gli array ("arrayFilters")
        update_on_wines.filterArray(identifier:"g.name", grapeName);
        update_on_styles.filterArray(identifier:"g.name", grapeName);

        // Applica l'update su tutti i documenti che contengono quella grape
        mongoTemplate.updateMulti(query_on_wines, update_on_wines, collectionName:"wines");
        mongoTemplate.updateMulti(query_on_styles, update_on_styles, collectionName:"styles");
    }

    System.out.println(x:"--- INFO: Grapes count updated successfully.\n\n");
}

```

Figure 4.23: updateWinesCountInGrapesPerWine()

6. A second `$group` aggregates the data per region, pushing the vintage details into an array.
7. `$project` slices the array to retain only the top 10 vintages per region.
8. A final `$project` renames fields for clarity and consistency, preparing documents with fields `name` (region name) and `top_10_vintages_of_the_week`.
9. `$merge` writes the output into the `regions` collection, updating existing entries by matching on `name`, and discarding any unmatched records.

Additionally, the method ensures that a unique index on `name` in the `regions` collection is in place to support this pipeline. This aggregation ensures that each region document reflects the current top 10 most popular vintages based on user reviews, enabling efficient retrieval and display of trending wines by area.

```

@Scheduled(cron = "0 0 2 * * MON") // Ogni lunedì alle 2 di notte
protected void updateTop10VintagesPerRegion() {
    System.out.println(x:"--- INFO: Declaring aggregation pipeline stages for collection \"regions\".");
    List<Document> pipeline = Arrays.asList(
        new Document(key:"$lookup",
            new Document(key:"from", value:"wines")
                .append(key:"localField", value:"wine_id.id")
                .append(key:"foreignField", value:"_id")
                .append(key:"as", value:"result")),
        new Document(key:"$unwind",
            new Document(key:"path", value:"$result")),
        new Document(key:"$project",
            new Document(key:"_id", value:$0L)
                .append(key:"wine_id", value:"$wine_id.id")
                .append(key:"wine_name", value:"$wine_id.name")
                .append(key:"wine_image", value:"$wine_id.image")
                .append(key:"year", value:"$wine_id.year")
                .append(key:"region", value:"$result.region.name")),
        new Document(key:"$group",
            new Document(key:"_id",
                new Document(key:"wine_id", value:"$wine_id")
                    .append(key:"year", value:"$year"))
                .append(key:"wine_name",
                    new Document(key:"$first", value:"$wine_name"))
                .append(key:"wine_image",
                    new Document(key:"$first", value:"$wine_image"))
                .append(key:"region",
                    new Document(key:"$first", value:"$region"))
                .append(key:"count",
                    new Document(key:"$sum", value:1))),
        new Document(key:"$sort",
            new Document(key:"count", -1)),
        new Document(key:"$group",
            new Document(key:"_id", value:"$region")
                .append(key:"vintages",
                    new Document(key:"$push",
                        new Document(key:"wine_id", value:"$._id.wine_id")
                            .append(key:"wine_name", value:"$._id.wine_name")
                            .append(key:"year", value:"$._id.year")
                            .append(key:"bottle", value:"$._id.image")
                            .append(key:"count", value:"$count"))))),
        new Document(key:"$project",
            new Document(key:"_id", value:$0L)
                .append(key:"name", value:"$._id")
                .append(key:"vintages",
                    new Document(key:"$slice", Arrays.asList(...a:"$vintages", 10L)))),
        new Document(key:"$project", new Document(key:"_id", value:$0)
            .append(key:"name", value:1)
            .append(key:"top_10_vintages_of_the_week", value:"$vintages")),
        new Document(key:"$merge", new Document(key:"into", value:"regions")
            .append(key:"on", value:"name")
            .append(key:"whenMatched", value:"merge")
            .append(key:"whenNotMatched", value:"discard"))));
    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(collectionName:"regions")
        .ensureIndex(
            new Index()
                .on(key:"name", Sort.Direction.ASC)
                .named(name:"on_name_UNIQUE")
                .unique()
        );
    System.out.println(x:"--- INFO: Ensured UNIQUE index on field \"name\" for collection \"regions\".");
    System.out.println(x:"--- INFO: Executing aggregation pipeline for collection \"regions\".");
    mongoTemplate.getCollection(collectionName:"reviews").aggregate(pipeline).allowDiskUse(allowDiskUse:true).toCollection();
    System.out.println(x:"--- INFO: Collection \"regions\" updated successfully.\n\n");
}

```

Figure 4.24: updateTop10VintagesPerRegion()

Weekly Update of Top 100 Vintages per Country

This aggregation pipeline is scheduled to run every Monday at 2:00 AM. It computes the top 100 most reviewed wine vintages for each country and updates the `countries` collection accordingly. Each vintage is defined as a unique pair of wine and year. The pipeline works on the `reviews` collection and consists of the following stages:

1. `$lookup` joins reviews with wines data by matching wine ids, adding information about the wine country.
2. `$unwind` flattens the resulting array from the join.
3. `$project` retains only the relevant fields: wine ID, name, image, year, and country.
4. `$group` aggregates by wine ID and vintage year counting the number of reviews for each unique combination.
5. `$sort` ranks these combinations by descending review count.
6. A second `$group` collects vintage information for each country into an array.
7. `$project` slices this array to retain only the top 100 vintages.
8. Another `$project` renames the fields to store the data as `name` (country) and `top_100_vintages_of_the_week`.
9. `$merge` writes the results into the `countries` collection, merging on the `name` field and discarding non-matching documents.

Additionally, the method ensures that a unique index on `name` in the `countries` collection is in place to support this pipeline. This process ensures that each document in the `countries` collection accurately reflects the most popular wine vintages in that country based on user activity, enabling high-level insights into national trends in wine appreciation.


```

@Scheduled(cron = "0 0 2 * * MON") // Ogni lunedì alle 2 di notte
protected void updateTop100VintagesPerCountry() {
    System.out.println(x:"--- INFO: Declaring aggregation pipeline stages for collection \"countries\".");
    List<Document> pipeline = Arrays.asList(
        new Document(key:"$lookup",
            new Document(key:"from", value:"wines")
                .append(key:"localField", value:"wine_id.id")
                .append(key:"foreignField", value:"_id")
                .append(key:"as", value:"result")),

        new Document(key:"$unwind",
            new Document(key:"path", value:"$result")),

        new Document(key:"$project",
            new Document(key:"_id", value:0L)
                .append(key:"wine_id", value:"$wine_id.id")
                .append(key:"wine_name", value:"$wine_id.name")
                .append(key:"wine_image", value:"$wine_id.image")
                .append(key:"year", value:"$wine_id.year")
                .append(key:"country", value:"$result.region.country.name")),

        new Document(key:"$group",
            new Document(key:"_id",
                new Document(key:"wine_id", value:"$wine_id")
                    .append(key:"year", value:"$year"))
                .append(key:"wine_name",
                    new Document(key:"$first", value:"$wine_name"))
                .append(key:"wine_image",
                    new Document(key:"$first", value:"$wine_image"))
                .append(key:"country",
                    new Document(key:"$first", value:"$country"))
                .append(key:"count",
                    new Document(key:"$sum", value:1L))),

        new Document(key:"$sort",
            new Document(key:"count", -1L)),

        new Document(key:"$group",
            new Document(key:"_id", value:"$country")
                .append(key:"vintages",
                    new Document(key:"$push",
                        new Document(key:"wine_id", value:"$._id.wine_id")
                            .append(key:"wine_name", value:"$._id.wine_name")
                            .append(key:"year", value:"$._id.year")
                            .append(key:"bottle", value:"$._id.wine_image")
                            .append(key:"count", value:"$count"))))),

        new Document(key:"$project",
            new Document(key:"_id", value:0L)
                .append(key:"name", value:"$_id")
                .append(key:"vintages",
                    new Document(key:"$slice", Arrays.asList(...:"$vintages", 100L)))),

        new Document(key:"$project", new Document(key:"_id", value:0)
            .append(key:"name", value:1)
            .append(key:"top_100_vintages_of_the_week", value:"$vintages")),

        new Document(key:"$merge", new Document(key:"into", value:"countries")
            .append(key:"on", value:"name")
            .append(key:"whenMatched", value:"merge")
            .append(key:"whenNotMatched", value:"discard"))));

    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(collectionName:"countries")
        .ensureIndex(
            new Index()
                .on(key:"name", Sort.Direction.ASC)
                .named(name:"on_name_UNIQUE")
                .unique()
        );
    System.out.println(x:"--- INFO: Ensured UNIQUE index on field \"name\" for collection \"countries\".");

    System.out.println(x:"--- INFO: Executing aggregation pipeline for collection \"countries\"...");
    mongoTemplate.getCollection(collectionName:"reviews").aggregate(pipeline).allowDiskUse(allowDiskUse:true).toCollection();

    System.out.println(x:"--- INFO: Collection \"countries\" updated successfully.\n\n");
}

```

Figure 4.25: updateTop100VintagesPerCountry()

In order to provide personalized and dynamic wine recommendations, we defined a scheduled aggregation that updates the `wine_tips` field for each user every Monday at 3:00 AM. The aggregation pipeline begins by projecting the necessary fields from the `users` collection: the user’s username and the list of their favorite wines (stored under `wine_favorites.id`). A first `$lookup` is then performed on the `reviews` collection to extract the top 3 highest-rated wines reviewed by the user, sorted by rating and review timestamp. These are combined with the user’s favorite wines using a `$setUnion` operation to form a comprehensive set of relevant wines for the user. The pipeline then unwinds this combined list and retrieves detailed information for each wine via a `$lookup` on the `wines` collection, focusing on extracting the wine’s style (`style.name`). For each user-style pair, a representative wine is selected using `$group` and `$first`, and another `$lookup` is used to retrieve up to 25 highly rated wines of the same style. These are sorted by descending average rating and rating count, and enriched with relevant metadata such as name, type, average price, image, and creation date. The resulting tips are then aggregated per user and stored as a list in the `wine_tips` field. To maintain a manageable size, this list is limited to 100 items per user using the `$slice` operator. Finally, the results are merged back into the original `users` collection using a `$merge` stage that updates existing documents based on the `login.username` field. To ensure the efficiency of this aggregation,

[illegible]

Figure 4.26: `updateWineTipsPerUser()`: part 1.

```

275     System.out.println("---- INFO: Ensured UNIQUE index on field \"username\" for collection \"\" + SOURCE_COLLECTION_NAME + "\".");
276
277     mongoTemplate
278         .indexOps(collectionName:"reviews")
279         .ensureIndex(
280             new Index()
281                 .on(key:"user_id.username", Sort.Direction.ASC)
282                 .on(key:"rating", Sort.Direction.DESC)
283                 .on(key:"created_at", Sort.Direction.DESC)
284                 .named(name:"index_for_wine_tips_and_new_wine_tips")
285         );
286     System.out.println(x:"---- INFO: Ensured index on fields \"user_id.username\", \"rating\" and \"created_at\" for collection \"reviews\".");
287
288     mongoTemplate
289         .indexOps(collectionName:"wines")
290         .ensureIndex(
291             new Index()
292                 .on(key:"style.name", Sort.Direction.ASC)
293                 .on(key:"statistics.ratings_average", Sort.Direction.DESC)
294                 .on(key:"statistics.ratings_count", Sort.Direction.DESC)
295                 .named(name:"index_for_wine_tips")
296         );
297     System.out.println(x:"---- INFO: Ensured index on fields \"style.name\", \"statistics.ratings_average\" and \"statistics.ratings_count\" for collection \"wines\".");
298     System.out.println(x:"---- INFO: Executing aggregation pipeline...");
299     mongoTemplate.getCollection(SOURCE_COLLECTION_NAME).aggregate(pipeline).toCollection();
300     System.out.println(x:"---- INFO: Field \"wine_tips\" in collection \"users\" updated successfully.\n\n");
301 }

```

Figure 4.27: updateWineTipsPerUser(): part 2.

proper indexes are enforced on the relevant fields in the **users**, **reviews**, and **wines** collections, including compound indexes on username, wine style, rating average, and rating count.

Updating New Wine Tips per User

This aggregation pipeline runs every Monday at 3:00 AM and aims to update the **new_wine_tips** field in the **users** collection. The goal is to generate personalized wine recommendations by combining each user's favorite wines and their top-rated recent reviews. The pipeline proceeds as follows:

- It starts by projecting each user's **username** and the IDs of their favorite wines.
- Then, a lookup stage fetches the top 3 wines reviewed by the user, prioritizing those with the highest rating and most recent review date.
- These two sets of wine IDs — favorites and top reviewed — are merged into a unique list to avoid duplicates.
- For each wine in this combined list, the pipeline retrieves detailed information from the **wines** collection, including the wine's style.
- Next, the documents are grouped by user and wine style, selecting one representative wine per style.
- Another lookup stage uses each wine style to find up to 25 recently created wines (within the last 6 months), sorted by creation date, including relevant data such as name, type, average rating, price, and image.
- The recommendations are then grouped per user, collecting up to 100 new wine tips.
- Finally, the pipeline merges these results back into the **users** collection, updating the **new_wine_tips** field only for existing users.

Additionally, the aggregation ensures the presence of the following indexes for performance optimization:

- A unique index on `login.username` in the `users` collection.
- A compound index on `user_id.username`, `rating`, and `created_at` in the `reviews` collection.
- An index on `style.name` and `created_at` in the `wines` collection.

Figure 4.28: `updateNewWineTipsPerUser()`: part 1.

Figure 4.29: `updateNewWineTipsPerUser()`: part 2.

Weekly Update of Top Vintages by Our Quality/Price Ratio per Wine Type

This aggregation pipeline is scheduled to run every Monday at 2:00 AM. It calculates the top vintages ranked by a custom quality/price (QoP) formula for each wine type (e.g., red, white, rosé) and updates the `top_vintages_by_our_qop_per_type` collection accordingly.

The pipeline operates on the `wines` collection and includes the following stages:

1. `$unwind` separates vintages embedded within each wine document.
2. `$project` retains necessary fields such as wine name, type, winery name, vintage year, image, ratings count, average quality rating, price, and computes the QoP score using the formula:

$$\text{QoP} = \frac{e^{\text{ratings_average}} \times \sqrt{\text{ratings_count}}}{\ln(\text{price})}$$

3. `$sort` orders vintages by QoP and ratings count in descending order.
4. `$group` aggregates vintages by wine type to determine the maximum QoP value per type and collects all vintages.
5. `$unwind` flattens vintages again to assign the max QoP for each type to every vintage.
6. `$addFields` adds a `points` field, representing each vintage's QoP as a percentage of the max QoP within its wine type, rounded to one decimal place.
7. `$group` regroups vintages by wine type, assembling the vintages with their respective points.
8. `$project` formats the output by excluding the internal ID, renaming the wine type, and limiting vintages to the top 1000 entries.
9. `$merge` updates the `top_vintages_by_our_qop_per_type` collection by replacing existing documents with matching wine types or inserting new ones.

Before running the pipeline, the method ensures that appropriate indexes exist for efficient aggregation and merging. This process maintains an up-to-date materialized view of the highest-ranked vintages per wine type, facilitating data-driven quality and price analysis for wine recommendations.

[illegible]

Figure 4.30: `updateTopVintagesByOurQoPPerType()`

```
private void createTopVintagesByOurQoPPerType() {
    if (!mongoTemplate.collectionExists(TOP_VINTAGES_OUR_QOP)) {
        mongoTemplate.createCollection(TOP_VINTAGES_OUR_QOP);
        System.out.println("--- INFO: Materialized view \"" + TOP_VINTAGES_OUR_QOP + "\" created successfully.");
    } else {
        System.out.println("--- INFO: Materialized view \"" + TOP_VINTAGES_OUR_QOP + "\" already exists.");
    }

    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(TOP_VINTAGES_OUR_QOP)
        .ensureIndex(
            new Index()
                .on(key:"type", Sort.Direction.ASC)
                .named(name:"on_type_UNIQUE")
                .unique()
        );
    System.out.println("--- INFO: Ensured UNIQUE index on field \"type\" for materialized view \"" + TOP_VINTAGES_OUR_QOP + "\".");
}
```

Figure 4.31: `createTopVintagesByOurQoPPerType()`

Weekly Update of Top Vintages by Base Quality/Price Ratio per Wine Type

This aggregation pipeline is scheduled to run every Monday at 2:00 AM. It calculates the top vintages ranked by a base quality/price (QoP) formula for each wine type (e.g., red, white, rosé) and updates the `top_vintages_by_qop_per_type` collection accordingly. The pipeline operates on the `wines` collection and includes the following

stages:

1. `$unwind` separates vintages embedded within each wine document.
2. `$project` retains necessary fields such as wine name, type, winery name, vintage year, image, ratings count, average quality rating, price, and computes the QoP score using the formula:

$$\text{QoP} = \frac{\text{ratings_average}}{\text{price}}$$

3. `$sort` orders vintages by QoP and ratings count in descending order.
4. `$group` aggregates vintages by wine type to determine the maximum QoP value per type and collects all vintages.
5. `$unwind` flattens vintages again to assign the max QoP for each type to every vintage.
6. `$addFields` adds a `points` field, representing each vintage's QoP as a percentage of the max QoP within its wine type, rounded to one decimal place.
7. `$group` regroups vintages by wine type, assembling the vintages with their respective points.
8. `$project` formats the output by excluding the internal ID, renaming the wine type, and limiting vintages to the top 1000 entries.
9. `$merge` updates the `top_vintages_by_qop_per_type` collection by replacing existing documents with matching wine types or inserting new ones.

Before running the pipeline, the method ensures that the `top_vintages_by_qop_per_type` materialized view exists by calling its creation routine. This process maintains an up-to-date materialized view of the highest-ranked vintages per wine type, enabling data-driven quality and price analysis for wine recommendations.

Weekly Update of Top Vintages by Average Rating per Wine Type

This aggregation pipeline is scheduled to run every Monday at 2:00 AM. It calculates a ranking of the top vintages based on their average ratings for each wine type (e.g., red, white, rosé) and updates the `top_vintages_by_ratings_per_type` collection accordingly. The pipeline operates on the `wines` collection and includes the following stages:

1. `$unwind` separates vintages embedded within each wine document.
2. `$project` retains necessary fields such as wine name, type, winery name, vintage year, price, image, average rating, and ratings count.

```

324 # @endnote[rows = "0 0 0 0 0 0"]
325 protected void updateForRatingAggQueryType() {
326   final String SOURCE_COLLECTION_NAME = "movies";
327   System.out.println("=== INFO: Defining aggregation pipeline stages for materialized view '"+ TOP_VINTAGES_QP + "'");
328   List<Document> pipeline = Arrays.asList(
329     new Document(key "$match",
330       new Document(key "$or", value "$ratings")),
331     new Document(key "$project",
332       new Document(key "$name", value "$_id"),
333       append(key "$type", value "$_id"),
334       append(key "$year", value "$primary.year"),
335       append(key "$year", value "$ratings.year"),
336       append(key "$img", value "$ratings.image"),
337       append(key "$ratings_count", value "$ratings.statistics.ratings_count"),
338       append(key "$quality", value "$ratings.statistics.ratings_average"),
339       append(key "$price", value "$ratings.price"),
340       append(key "$qp",
341         new Document(key "$divide", Arrays.asList(... "$ratings.statistics.ratings_average", "$ratings.price")))),
342     new Document(key "$sort",
343       new Document(key "$qp", $)),
344     append(key "$ratings_count", -1)),
345     new Document(key "$group",
346       append(key "$_id", value "$type"),
347       append(key "$max_qp",
348         new Document(key "$sum", value "$qp"))),
349     append(key "$vintages",
350       new Document(key "$name",
351         append(key "$name", value "$name"),
352         append(key "$year", value "$primary"),
353         append(key "$year", value "$year"),
354         append(key "$img", value "$img"),
355         append(key "$ratings_count", value "$ratings_count"),
356         append(key "$quality", value "$quality"),
357         append(key "$price", value "$price"),
358         append(key "$qp", value "$qp"))),
359     new Document(key "$match",
360       new Document(key "$path", value "$vintages")),
361     new Document(key "$indexFields",
362       new Document(key "$vintages.primary",
363         new Document(key "$name", Arrays.asList(new Document(key "$multiplier", Arrays.asList(... $INDEX,
364           new Document(key "$divide", Arrays.asList(... "$vintages.qp", "$max_qp"))), $)),
365       new Document(key "$form",
366         new Document(key "$_id", value "$_id"),
367         append(key "$vintages",
368           new Document(key "$name",
369             new Document(key "$name", value "$vintages.name"),
370             append(key "$year", value "$vintages.year"),
371             append(key "$img", value "$vintages.img"),
372             append(key "$ratings_count", value "$vintages.ratings_count"),
373             append(key "$quality", value "$vintages.quality"),
374             append(key "$price", value "$vintages.price"),
375             append(key "$points", value "$vintages.points"))),
376       new Document(key "$form",
377         new Document(key "$_id", value "$_id"),
378         append(key "$type", value "$type"),
379         append(key "$year", value "$year"),
380         append(key "$img", value "$img"),
381         append(key "$ratings_count", value "$ratings_count"),
382         append(key "$quality", value "$quality"),
383         append(key "$price", value "$price"),
384         append(key "$points", value "$points")),
385     new Document(key "$form",
386       new Document(key "$_id", value "$_id"),
387       append(key "$type", value "$type"),
388       append(key "$year", value "$year"),
389       append(key "$img", value "$img"),
390       append(key "$ratings_count", value "$ratings_count"),
391       append(key "$quality", value "$quality"),
392       append(key "$price", value "$price"),
393       append(key "$points", value "$points")),
394     new Document(key "$form",
395       new Document(key "$_id", value "$_id"),
396       append(key "$type", value "$type"),
397       append(key "$year", value "$year"),
398       append(key "$img", value "$img"),
399       append(key "$ratings_count", value "$ratings_count"),
400       append(key "$quality", value "$quality"),
401       append(key "$price", value "$price"),
402       append(key "$points", value "$points")),
403     new Document(key "$form",
404       new Document(key "$_id", value "$_id"),
405       append(key "$type", value "$type"),
406       append(key "$year", value "$year"),
407       append(key "$img", value "$img"),
408       append(key "$ratings_count", value "$ratings_count"),
409       append(key "$quality", value "$quality"),
410       append(key "$price", value "$price"),
411       append(key "$points", value "$points")),
412     new Document(key "$form",
413       new Document(key "$_id", value "$_id"),
414       append(key "$type", value "$type"),
415       append(key "$year", value "$year"),
416       append(key "$img", value "$img"),
417       append(key "$ratings_count", value "$ratings_count"),
418       append(key "$quality", value "$quality"),
419       append(key "$price", value "$price"),
420       append(key "$points", value "$points")),
421     new Document(key "$form",
422       new Document(key "$_id", value "$_id"),
423       append(key "$type", value "$type"),
424       append(key "$year", value "$year"),
425       append(key "$img", value "$img"),
426       append(key "$ratings_count", value "$ratings_count"),
427       append(key "$quality", value "$quality"),
428       append(key "$price", value "$price"),
429       append(key "$points", value "$points")),
430     new Document(key "$form",
431       new Document(key "$_id", value "$_id"),
432       append(key "$type", value "$type"),
433       append(key "$year", value "$year"),
434       append(key "$img", value "$img"),
435       append(key "$ratings_count", value "$ratings_count"),
436       append(key "$quality", value "$quality"),
437       append(key "$price", value "$price"),
438       append(key "$points", value "$points")),
439     new Document(key "$form",
440       new Document(key "$_id", value "$_id"),
441       append(key "$type", value "$type"),
442       append(key "$year", value "$year"),
443       append(key "$img", value "$img"),
444       append(key "$ratings_count", value "$ratings_count"),
445       append(key "$quality", value "$quality"),
446       append(key "$price", value "$price"),
447       append(key "$points", value "$points")),
448     new Document(key "$form",
449       new Document(key "$_id", value "$_id"),
450       append(key "$type", value "$type"),
451       append(key "$year", value "$year"),
452       append(key "$img", value "$img"),
453       append(key "$ratings_count", value "$ratings_count"),
454       append(key "$quality", value "$quality"),
455       append(key "$price", value "$price"),
456       append(key "$points", value "$points")),
457     new Document(key "$form",
458       new Document(key "$_id", value "$_id"),
459       append(key "$type", value "$type"),
460       append(key "$year", value "$year"),
461       append(key "$img", value "$img"),
462       append(key "$ratings_count", value "$ratings_count"),
463       append(key "$quality", value "$quality"),
464       append(key "$price", value "$price"),
465       append(key "$points", value "$points")),
466     new Document(key "$form",
467       new Document(key "$_id", value "$_id"),
468       append(key "$type", value "$type"),
469       append(key "$year", value "$year"),
470       append(key "$img", value "$img"),
471       append(key "$ratings_count", value "$ratings_count"),
472       append(key "$quality", value "$quality"),
473       append(key "$price", value "$price"),
474       append(key "$points", value "$points")),
475     new Document(key "$form",
476       new Document(key "$_id", value "$_id"),
477       append(key "$type", value "$type"),
478       append(key "$year", value "$year"),
479       append(key "$img", value "$img"),
480       append(key "$ratings_count", value "$ratings_count"),
481       append(key "$quality", value "$quality"),
482       append(key "$price", value "$price"),
483       append(key "$points", value "$points")),
484     new Document(key "$form",
485       new Document(key "$_id", value "$_id"),
486       append(key "$type", value "$type"),
487       append(key "$year", value "$year"),
488       append(key "$img", value "$img"),
489       append(key "$ratings_count", value "$ratings_count"),
490       append(key "$quality", value "$quality"),
491       append(key "$price", value "$price"),
492       append(key "$points", value "$points")),
493     new Document(key "$form",
494       new Document(key "$_id", value "$_id"),
495       append(key "$type", value "$type"),
496       append(key "$year", value "$year"),
497       append(key "$img", value "$img"),
498       append(key "$ratings_count", value "$ratings_count"),
499       append(key "$quality", value "$quality"),
500       append(key "$price", value "$price"),
501       append(key "$points", value "$points")),
502     new Document(key "$form",
503       new Document(key "$_id", value "$_id"),
504       append(key "$type", value "$type"),
505       append(key "$year", value "$year"),
506       append(key "$img", value "$img"),
507       append(key "$ratings_count", value "$ratings_count"),
508       append(key "$quality", value "$quality"),
509       append(key "$price", value "$price"),
510       append(key "$points", value "$points")),
511     new Document(key "$form",
512       new Document(key "$_id", value "$_id"),
513       append(key "$type", value "$type"),
514       append(key "$year", value "$year"),
515       append(key "$img", value "$img"),
516       append(key "$ratings_count", value "$ratings_count"),
517       append(key "$quality", value "$quality"),
518       append(key "$price", value "$price"),
519       append(key "$points", value "$points")),
520     new Document(key "$form",
521       new Document(key "$_id", value "$_id"),
522       append(key "$type", value "$type"),
523       append(key "$year", value "$year"),
524       append(key "$img", value "$img"),
525       append(key "$ratings_count", value "$ratings_count"),
526       append(key "$quality", value "$quality"),
527       append(key "$price", value "$price"),
528       append(key "$points", value "$points")),
529     new Document(key "$form",
530       new Document(key "$_id", value "$_id"),
531       append(key "$type", value "$type"),
532       append(key "$year", value "$year"),
533       append(key "$img", value "$img"),
534       append(key "$ratings_count", value "$ratings_count"),
535       append(key "$quality", value "$quality"),
536       append(key "$price", value "$price"),
537       append(key "$points", value "$points")),
538     new Document(key "$form",
539       new Document(key "$_id", value "$_id"),
540       append(key "$type", value "$type"),
541       append(key "$year", value "$year"),
542       append(key "$img", value "$img"),
543       append(key "$ratings_count", value "$ratings_count"),
544       append(key "$quality", value "$quality"),
545       append(key "$price", value "$price"),
546       append(key "$points", value "$points")),
547     new Document(key "$form",
548       new Document(key "$_id", value "$_id"),
549       append(key "$type", value "$type"),
550       append(key "$year", value "$year"),
551       append(key "$img", value "$img"),
552       append(key "$ratings_count", value "$ratings_count"),
553       append(key "$quality", value "$quality"),
554       append(key "$price", value "$price"),
555       append(key "$points", value "$points")),
556     new Document(key "$form",
557       new Document(key "$_id", value "$_id"),
558       append(key "$type", value "$type"),
559       append(key "$year", value "$year"),
560       append(key "$img", value "$img"),
561       append(key "$ratings_count", value "$ratings_count"),
562       append(key "$quality", value "$quality"),
563       append(key "$price", value "$price"),
564       append(key "$points", value "$points")),
565     new Document(key "$form",
566       new Document(key "$_id", value "$_id"),
567       append(key "$type", value "$type"),
568       append(key "$year", value "$year"),
569       append(key "$img", value "$img"),
570       append(key "$ratings_count", value "$ratings_count"),
571       append(key "$quality", value "$quality"),
572       append(key "$price", value "$price"),
573       append(key "$points", value "$points")),
574     new Document(key "$form",
575       new Document(key "$_id", value "$_id"),
576       append(key "$type", value "$type"),
577       append(key "$year", value "$year"),
578       append(key "$img", value "$img"),
579       append(key "$ratings_count", value "$ratings_count"),
580       append(key "$quality", value "$quality"),
581       append(key "$price", value "$price"),
582       append(key "$points", value "$points")),
583     new Document(key "$form",
584       new Document(key "$_id", value "$_id"),
585       append(key "$type", value "$type"),
586       append(key "$year", value "$year"),
587       append(key "$img", value "$img"),
588       append(key "$ratings_count", value "$ratings_count"),
589       append(key "$quality", value "$quality"),
590       append(key "$price", value "$price"),
591       append(key "$points", value "$points")),
592     new Document(key "$form",
593       new Document(key "$_id", value "$_id"),
594       append(key "$type", value "$type"),
595       append(key "$year", value "$year"),
596       append(key "$img", value "$img"),
597       append(key "$ratings_count", value "$ratings_count"),
598       append(key "$quality", value "$quality"),
599       append(key "$price", value "$price"),
600       append(key "$points", value "$points")),
601     new Document(key "$form",
602       new Document(key "$_id", value "$_id"),
603       append(key "$type", value "$type"),
604       append(key "$year", value "$year"),
605       append(key "$img", value "$img"),
606       append(key "$ratings_count",
```

Figure 4.32: `updateTopVintagesByQoPPerType()`

```
private void createTopVintagesByQoPPerType() {
    if (!mongoTemplate.collectionExists(TOP_VINTAGES_QOP)) {
        mongoTemplate.createCollection(TOP_VINTAGES_QOP);
        System.out.println("--- INFO: Materialized view \"" + TOP_VINTAGES_QOP + "\" created successfully.");
    } else {
        System.out.println("--- INFO: Materialized view \"" + TOP_VINTAGES_QOP + "\" already exists.");
    }

    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(TOP_VINTAGES_QOP)
        .ensureIndex(
            new Index()
                .on(key:"type", Sort.Direction.ASC)
                .named(name:"on_type_UNIQUE")
                .unique()
        );
    System.out.println("--- INFO: Ensured UNIQUE index on field \"type\" for materialized view \"" + TOP_VINTAGES_QOP + "\".");
}
```

Figure 4.33: `createTopVintagesByQoPPerType()`

3. `$sort` orders vintages by average rating and ratings count in descending order.
4. `$group` aggregates vintages by wine type, collecting all vintages to form the top vintages list per type.
5. `$project` formats the output by excluding the internal ID, renaming the wine type, and limiting vintages to the top 1000 entries.
6. `$merge` updates the `top_vintages_by_ratings_per_type` collection by replacing existing documents with matching wine types or inserting new ones.

Before running the pipeline, the method ensures that the `top_vintages_by_ratings_per_type` materialized view exists by calling its creation routine. This process maintains an up-to-date materialized view of the highest-ranked vintages per wine type based on average ratings, supporting quality-focused wine recommendations.


```

@Scheduled(cron = "0 0 2 * * MON") // Ogni lunedì alle 2 di notte
protected void updateTopVintagesByRatingsPerType() {
    final String SOURCE_COLLECTION_NAME = "wines";
    System.out.println("--- INFO: Declaring aggregation pipeline stages for materialized view \"\" + TOP_VINTAGES_RATINGS + "\".");
    List<Document> pipeline = Arrays.asList(
        new Document(key:"$unwind",
            new Document(key:"path", value:"$vintages")),
        new Document(key:"$project",
            new Document(key:"name", value:1L)
                .append(key:"type", value:1L)
                .append(key:"winery", value:"$winery.name")
                .append(key:"year", value:"$vintages.year")
                .append(key:"price", value:"$vintages.price")
                .append(key:"image", value:"$vintages.image")
                .append(key:"ratings_average", value:"$vintages.statistics.ratings_average")
                .append(key:"ratings_count", value:"$vintages.statistics.ratings_count")),
        new Document(key:"$sort",
            new Document(key:"ratings_average", -1L)
                .append(key:"ratings_count", -1L)),
        new Document(key:"$group",
            new Document(key:"_id", value:"$type")
                .append(key:"vintages",
                    new Document(key:"$push",
                        new Document(key:"wine", value:"$name")
                            .append(key:"winery", value:"$winery")
                            .append(key:"year", value:"$year")
                            .append(key:"price", value:"$price")
                            .append(key:"image", value:"$image")
                            .append(key:"ratings_average", value:"$ratings_average")
                            .append(key:"ratings_count", value:"$ratings_count")))),
            new Document(key:"$project",
                new Document(key:"_id", value:0L)
                    .append(key:"type", value:"$_id")
                    .append(key:"vintages",
                        new Document(key:"$slice", Arrays.asList(...:"$vintages", 1000L))))),
        new Document(key:"$merge",
            new Document(key:"into", TOP_VINTAGES_RATINGS)
                .append(key:"on", value:"type")
                .append(key:"whenMatched", value:"replace")
                .append(key:"whenNotMatched", value:"insert"))
    );

    // Creating the materialized view
    System.out.println("--- INFO: Creating empty materialized view \"\" + TOP_VINTAGES_RATINGS + "\".");
    createTopVintagesByRatingsPerType();

    // Executing the pipeline
    System.out.println("--- INFO: Executing aggregation pipeline for materialized view \"\" + TOP_VINTAGES_RATINGS + "\".");
    mongoTemplate.getCollection(SOURCE_COLLECTION_NAME).aggregate(pipeline).allowDiskUse(allowDiskUse:true).toCollection();

    System.out.println("--- INFO: Materialized view \"\" + TOP_VINTAGES_RATINGS + "\" updated successfully.\n\n");
}

```

Figure 4.34: updateTopVintagesByRatingsPerType()

```

private void createTopVintagesByRatingsPerType() {
    if (!mongoTemplate.collectionExists(TOP_VINTAGES_RATINGS)) {
        mongoTemplate.createCollection(TOP_VINTAGES_RATINGS);
        System.out.println("--- INFO: Materialized view \"\" + TOP_VINTAGES_RATINGS + "\" created successfully.");
    } else {
        System.out.println("--- INFO: Materialized view \"\" + TOP_VINTAGES_RATINGS + "\" already exists.");
    }

    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(TOP_VINTAGES_RATINGS)
        .ensureIndex(
            new Index()
                .on(key:"type", Sort.Direction.ASC)
                .named(name:"on_type_UNIQUE")
                .unique()
        );
    System.out.println("--- INFO: Ensured UNIQUE index on field \"type\" for materialized view \"\" + TOP_VINTAGES_RATINGS + "\".");
}

```

Figure 4.35: createTopVintagesByRatingsPerType()

Weekly Update of Top Wines by Average Rating per Wine Type

This aggregation pipeline is executed every Monday at 2:00 AM. It generates a ranking of the top wines based on their overall average rating, categorized by wine type (e.g., red, white, rosé). The final result is stored in the `top_wines_by_ratings_per_type` materialized view. The aggregation is performed on the `wines` collection and includes the following stages:

1. `$project` extracts the wine name, type, winery name, and overall average statistics. It also computes the average price across vintages (rounded to two decimal places), and selects the first available image from the list of vintages.
2. `$sort` orders wines by descending average rating and ratings count, giving priority to highly rated and frequently reviewed wines.
3. `$group` aggregates wines by type and collects the relevant fields into a list named `wines` for each category.
4. `$project` prepares the final output by renaming the wine type field and limiting the number of top wines to 1000 per type.
5. `$merge` updates the `top_wines_by_ratings_per_type` materialized view, replacing existing documents where the type matches or inserting new entries otherwise.

Before executing the pipeline, the method ensures that the `top_wines_by_ratings_per_type` collection exists and is properly initialized. This process guarantees an up-to-date and efficient lookup structure for the highest-rated wines of each type, enabling fast access to top-quality selections.

Weekly Update of Top Wineries by Average Wine Ratings

This aggregation pipeline is executed every Monday at 2:00 AM. It generates a unified ranking of the top wineries based on the average rating of all their wines, regardless of wine type. The final result is stored in the `top_wineries_by_wines_ratings` materialized view. The aggregation is performed on the `wines` collection and includes the following stages:

1. `$group` aggregates all wines by the winery's username, calculating the average rating and the total number of ratings across all wines produced by each winery.
2. `$lookup` joins the result with the `wineries` collection to retrieve the full winery information using the username as a key.
3. `$unwind` flattens the resulting array from the previous join to access the embedded winery document.

```

@Scheduled(cron = "0 0 2 * * MON") // Ogni lunedì alle 2 di notte
protected void updateTopWinesByRatingsPerType() {
    final String SOURCE_COLLECTION_NAME = "wines";
    System.out.println("--- INFO: Declaring aggregation pipeline stages for materialized view \"\" + TOP_WINES_RATINGS + "\".");
    List<Document> pipeline = Arrays.asList(
        new Document(key: "$project",
            new Document(key: "name", value: 1L)
                .append(key: "type", value: 1L)
                .append(key: "winery", value: "$winery.name")
                .append(key: "prices_average", // dato che i prezzi sono diversi per ogni vintage, calcolo la media dei prezzi delle annate
                    new Document(key: "$round",
                        Arrays.asList(new Document(key: "$avg", value: "$vintages.price"), 2L)))
                .append(key: "image",
                    new Document(key: "$arrayElemAt", Arrays.asList(...: "$vintages.image", 0L)))
                .append(key: "ratings_average", value: "$statistics.ratings_average")
                .append(key: "ratings_count", value: "$statistics.ratings_count")),
        new Document(key: "$sort",
            new Document(key: "ratings_average", -1L)
                .append(key: "ratings_count", -1L)),
        new Document(key: "$group",
            new Document(key: "_id", value: "$type")
                .append(key: "wines",
                    new Document(key: "$push",
                        new Document(key: "wine", value: "$name")
                            .append(key: "winery", value: "$winery")
                            .append(key: "image", value: "$image")
                            .append(key: "prices_average", value: "$prices_average")
                            .append(key: "ratings_average", value: "$ratings_average")
                            .append(key: "ratings_count", value: "$ratings_count"))))),
        new Document(key: "$project",
            new Document(key: "_id", value: 0L)
                .append(key: "type", value: "$_id")
                .append(key: "wines",
                    new Document(key: "$slice", Arrays.asList(...: "$wines", 1000L)))),
        new Document(key: "$merge",
            new Document(key: "into", TOP_WINES_RATINGS)
                .append(key: "on", value: "type")
                .append(key: "whenMatched", value: "replace")
                .append(key: "whenNotMatched", value: "insert"))
    );

    // Creating the materialized view
    System.out.println("--- INFO: Creating empty materialized view \"\" + TOP_WINES_RATINGS + "\"...");
    createTopWinesByRatingsPerType();

    // Executing the pipeline
    System.out.println("--- INFO: Executing aggregation pipeline for materialized view \"\" + TOP_WINES_RATINGS + "\"...");
    mongoTemplate.getCollection(SOURCE_COLLECTION_NAME).aggregate(pipeline).allowDiskUse(allowDiskUse: true).toCollection();

    System.out.println("--- INFO: Materialized view \"\" + TOP_WINES_RATINGS + "\" updated successfully.\n\n");
}

```

Figure 4.36: updateTopWinesByRatingsPerType()

```

private void createTopWinesByRatingsPerType() {
    if (!mongoTemplate.collectionExists(TOP_WINES_RATINGS)) {
        mongoTemplate.createCollection(TOP_WINES_RATINGS);
        System.out.println("--- INFO: Materialized view \"\" + TOP_WINES_RATINGS + "\" created successfully.");
    } else {
        System.out.println("--- INFO: Materialized view \"\" + TOP_WINES_RATINGS + "\" already exists.");
    }

    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(TOP_WINES_RATINGS)
        .ensureIndex(
            new Index()
                .on(key: "type", Sort.Direction.ASC)
                .named(name: "on_type_UNIQUE")
                .unique()
        );
    System.out.println("--- INFO: Ensured UNIQUE index on field \"type\" for materialized view \"\" + TOP_WINES_RATINGS + "\".");
}

```

Figure 4.37: createTopWinesByRatingsPerType()

4. `$sort` orders wineries by descending average rating and ratings count, prioritizing both quality and popularity.
5. `$project` reshapes the document to include only relevant fields such as winery name, location, thumbnail image, and the aggregated statistics, rounding the

average rating to one decimal place.

6. `$merge` writes the final output to the `top_wineries_by_wines_ratings` materialized view, replacing existing entries or inserting new ones based on the winery username.

```
@Scheduled(cron = "0 0 2 * * MON") // Ogni lunedì alle 2 di notte
protected void updateTopWineriesByWineRatings() {
    final String SOURCE_COLLECTION_NAME = "wines";
    System.out.println("--- INFO: Declaring aggregation pipeline stages for materialized view \"" + TOP_WINERIES_RATINGS + "\".");
    List<Document> pipeline = Arrays.asList(
        new Document(key: "$group",
            new Document(key: "_id", value: "$winery.username")
                .append(key: "ratings_average",
                    new Document(key: "$avg", value: "$statistics.ratings_average"))
                .append(key: "ratings_count",
                    new Document(key: "$sum", value: "$statistics.ratings_count"))),
        new Document(key: "$lookup",
            new Document(key: "from", value: "wineries")
                .append(key: "localField", value: "_id")
                .append(key: "foreignField", value: "login.username")
                .append(key: "as", value: "winery")),
        new Document(key: "$unwind",
            new Document(key: "path", value: "$winery")),
        new Document(key: "$sort",
            new Document(key: "ratings_average", -1L)
                .append(key: "ratings_count", -1L)),
        new Document(key: "$project",
            new Document(key: "_id", value: 0L)
                .append(key: "winery_username", value: "$winery.login.username")
                .append(key: "thumbnail", value: "$winery.picture.thumbnail")
                .append(key: "winery", value: "$winery.name")
                .append(key: "region", value: "$winery.region")
                .append(key: "country", value: "$winery.country")
                .append(key: "ratings_average",
                    new Document(key: "$round", Arrays.asList(...: "$ratings_average", 1L)))
                .append(key: "ratings_count", value: "$ratings_count")),
        new Document(key: "$merge",
            new Document(key: "into", TOP_WINERIES_RATINGS)
                .append(key: "on", value: "winery_username")
                .append(key: "whenMatched", value: "replace")
                .append(key: "whenNotMatched", value: "insert"))
    );

    // Creating the materialized view
    System.out.println("--- INFO: Creating empty materialized view \"" + TOP_WINERIES_RATINGS + "\".");
    createTopWineriesByWineRatings();

    // Executing the pipeline
    System.out.println("--- INFO: Executing aggregation pipeline for materialized view \"" + TOP_WINERIES_RATINGS + "\".");
    mongoTemplate.getCollection(SOURCE_COLLECTION_NAME).aggregate(pipeline).allowDiskUse(allowDiskUse: true).toCollection();

    System.out.println("--- INFO: Materialized view \"" + TOP_WINERIES_RATINGS + "\" updated successfully.\n\n");
}
```

Figure 4.38: `updateTopWineriesByWineRatings()`

Before executing the pipeline, the method ensures that the `top_wineries_by_wines_ratings` collection exists and is properly initialized. This process provides a centralized, up-to-date leaderboard of wineries based on the quality of their wines, allowing users to easily discover the most highly rated producers.

```

private void createTopWineriesByWineRatings() {
    if (!mongoTemplate.collectionExists(TOP_WINERIES_RATINGS)) {
        mongoTemplate.createCollection(TOP_WINERIES_RATINGS);
        System.out.println("--- INFO: Materialized view \"" + TOP_WINERIES_RATINGS + "\" created successfully.");
    } else {
        System.out.println("--- INFO: Materialized view \"" + TOP_WINERIES_RATINGS + "\" already exists.");
    }

    // Assicuro che esistano gli indici NECESSARI alla pipeline.
    mongoTemplate
        .indexOps(TOP_WINERIES_RATINGS)
        .ensureIndex(
            new Index()
                .on(key:"winery_username", Sort.Direction.ASC)
                .named(name:"on_winery_username_UNIQUE")
                .unique()
        );
    System.out.println("--- INFO: Ensured UNIQUE index on field \"winery_username\" for materialized view \"" + TOP_WINERIES_RATINGS + "\".");

    // Assicuro che esistano gli indici FUNZIONALI alla pipeline.
    mongoTemplate
        .indexOps(collectionName:"wineries")
        .ensureIndex(
            new Index()
                .on(key:"login.username", Sort.Direction.ASC)
                .named(name:"on_username_UNIQUE")
                .unique()
        );
    System.out.println("x:--- INFO: Ensured UNIQUE index on field \"login.username\" for collection \"wineries\".");
}

```

Figure 4.39: createTopWineriesByWineRatings()

4.4 Neo4j Graph Queries

4.4.1 CRUD operations

CRUD operations have been implemented for each of the following entities in Neo4j: users, wineries, and reviews. To avoid redundancy, we will not detail all of them. It is important to note that these Neo4j operations are tightly synchronized with the corresponding MongoDB actions.

With regard to **reviews**:

- When a new review is added in MongoDB, it is immediately created in the Neo4j graph.
- When a review is updated in MongoDB, the change is promptly reflected in Neo4j, provided the review exists in the graph.
- When a review is deleted from MongoDB, it is also removed from the Neo4j graph if present.

The same synchronization logic applies to **users** and **wineries** entities, ensuring data consistency across both databases. In fact, with regard to users:

- When a new user is added in MongoDB, it is immediately created in the Neo4j graph.
- When a user is updated in MongoDB, the change is promptly reflected in Neo4j.

- When a user is deleted from MongoDB, it is also removed from the Neo4j graph if present.

With regard to wineries:

- When a new winery is added in MongoDB, it is immediately created in the Neo4j graph.
- When a winery is updated in MongoDB, the change is promptly reflected in Neo4j.
- When a winery is deleted from MongoDB, it is also removed from the Neo4j graph if present.

```
@Transactional
public Review addReview(String username, CreateReviewDTO createdReview) throws ResourceNotFoundException, ResourceAlreadyExistsException {
    // Controllo se l'utente esiste
    User user = userRepository.findByLogin_Username(username)
        .orElseThrow(() -> new ResourceNotFoundException("User with username " + username + " not found."));

    // Controllo se l'utente ha già recensito il vino
    if (reviewRepository.findByUserId_UsernameAndWineId_IdAndYear(username, createdReview.getWineId(), createdReview.getYear()).isPresent()) {
        throw new ResourceAlreadyExistsException("User with username " + username + " has already reviewed the wine with id " + createdReview.getWineId() + " and year " + createdReview.getYear() + ".");
    }

    Review review = new Review();
    review.setUserId(user.getId());
    review.setWineId(new WineId());

    review.getUserId().setUsername(user.getLogin().getUsername());
    review.getUserId().setThumbnail(user.getPicture().getThumbnail());
    review.getWineId().setId(createdReview.getWineId());
    review.getWineId().setYear(createdReview.getYear());
    review.setRating(createdReview.getRating());
    review.setText(createdReview.getText());

    // Devo controllare che l'utente abbia indicato un vino e un'annata esistenti
    Wine wine = wineRepository.findByIdAndVintage_Year(review.getWineId().getId(), review.getWineId().getYear()).orElseThrow(
        () -> new ResourceNotFoundException("Wine with id " + review.getWineId().getId() + " and year " + review.getWineId().getYear() + " not found."));

    review.getWineId().setImage(wine.getVintages().get(index:0).getImage());
    review.getWineId().setName(wine.getName());
    review.setLikesCount((long) 0);
    review.setDislikesCount((long) 0);
    review.setCreatedAt(Instant.now());

    // Setto l'id della recensione
    review.setId(idCounterService.generateSequence(sequenceName="reviews"));

    ReviewEmbedded reviewEmbedded = new ReviewEmbedded(review.getId(), review.getUserId(), review.getWineId(), review.getRating(), review.getText(), review.getCreatedAt(), review.getLikesCount(), review.getDislikesCount());

    // Aggiungo la recensione alla lista delle recensioni dell'utente (collection users)
    if (user.getReviews().size() >= 3) {
        user.getReviews().remove(index:2);
    }
    user.getReviews().add(index:0, reviewEmbedded);
    userRepository.save(user);

    Review savedReview = reviewRepository.save(review);

    // Sincronizzazione Neo4j
    reviewNeo4jRepository.createReviewForUser(
        username,
        user.getName().getFirst(),
        user.getName().getLast(),
        savedReview.getId(),
        savedReview.getText(),
        savedReview.getRating(),
        savedReview.getWineId().getId(),
        savedReview.getWineId().getName(),
        savedReview.getWineId().getYear(),
        savedReview.getWineId().getImage(),
        savedReview.getUserId().getThumbnail(),
        savedReview.getCreatedAt()
    );

    return savedReview;
}
```

Figure 4.40: `addReview()`: an example method to demonstrate that Neo4j is updated immediately after MongoDB.

4.4.2 Relevant Operations

User's feed creation

To retrieve the personalized feed, the application must call the following methods:

- `public @ResponseBody List<Map<String, Object>> getPersonalizedFeed(@RequestParam String username, @RequestParam(defaultValue = "1") int page, @RequestParam(defaultValue = "20") int size)`
- `public List<Map<String, Object>> getPersonalizedFeed(String username, int page, int pageSize)`
- `public List<Map<String, Object>> getPaginatedFeed(String username, int skip, int limit)`

The first method is a `@RestController` endpoint and is responsible for handling a GET request to the `/feed` path. It is secured so that only the user themselves or an administrator can access it. The controller delegates the logic to the service layer, which calculates the correct offset (`skip`) based on the page and page size, and then calls the repository method to fetch the data. The third method interacts directly with the Neo4j graph. It performs the following operations:

- Finds all the users followed by the given user.
- For each followed user, retrieves the most recent reviews (up to three).
- Collects the necessary review details (e.g., wine name, rating, date).
- Orders the results by date in descending order and applies pagination using `SKIP` and `LIMIT`.

This mechanism allows the system to provide each user with a feed that contains the latest reviews written by the users they follow, maintaining a social and personalized experience. The following figures show some of the code snippets involved in the process.

```
@GetMapping("/feed")
@Secured({ "ROLE_USER", "ROLE_ADMIN" })
@PreAuthorize("username == authentication.principal.username or hasRole('ROLE_ADMIN')")
public List<Map<String, Object>> getPersonalizedFeed(
    @RequestParam String username,
    @RequestParam(defaultValue = "1") int page,
    @RequestParam(defaultValue = "20") int size) {
    return reviewService.getPersonalizedFeed(username, page, size);
}
```

Figure 4.41: `@RestController` method handling the get user's feed request.

```
public List<Map<String, Object>> getPersonalizedFeed(String username, int page, int pageSize) {
    int skip = (page - 1) * pageSize;
    return reviewNeo4jRepository.getPaginatedFeed(username, skip, pageSize);
}
```

Figure 4.42: Service method to create user's feed.

```

public List<Map<String, Object>> getPaginatedFeed(String username, int skip, int limit) {
    String query = """
        MATCH (me:User {username: $username})-[:FOLLOWS]->(friend:User)
        MATCH (friend)-[:WROTE]->(review:Review)
        WITH friend, review
        ORDER BY review.created_at DESC
        WITH friend, collect(review)[0..3] AS recentReviews
        UNWIND recentReviews AS review
        RETURN {
            authorUsername: friend.username,
            authorThumbnail: friend.thumbnail,
            wineName: review.wineName,
            wineYear: review.wineYear,
            wineId: review.wineId,
            reviewText: review.text,
            reviewId: review.reviewId,
            rating: review.rating,
            wineImage: review.image,
            date: review.created_at
        } AS result
        ORDER BY result.date DESC
        SKIP $skip
        LIMIT $limit
    """,

    return new ArrayList<>() {
        neo4jClient.query(query)
            .bindAll(
                Map.of(
                    k1:"username", username,
                    k2:"skip", skip,
                    k3:"limit", limit
                )
            )
            .fetch()
            .all()
    };
}

```

Figure 4.43: Neo4j query (in `ReviewNeo4jRepository`) that retrieves up to three of the most recent reviews per followed user, sorted by creation date and paginated to build the personalized feed.

Get follow suggestions

To retrieve follow suggestions, the application must call the following methods:

- `public @ResponseBody List<Map<String, Object>> getSuggestedFollows(@RequestParam String username)`
- `public List<Map<String, Object>> getSuggestedFollows(String username)`
- `public List<Map<String, Object>> getSuggestedFollows(String username)`
- `public List<Map<String, Object>> getRandomFollows(String username, int limit)`

The first method is a `@RestController` endpoint that handles a GET request to the `/suggestions` path. It is protected so that only the user or an administrator can request suggestions. The controller delegates the logic to the service layer, which

calls the main repository method to retrieve follow suggestions. If fewer than 10 suggestions are returned, the service fetches additional random suggestions using a fallback method. The third method queries the Neo4j graph to identify second-degree connections (i.e., "friends of friends") who are not already followed by the user. It performs the following steps:

- Finds all the users followed by the target user.
- Looks for users or wineries followed by those friends but not yet followed by the user.
- Excludes the user themselves from the suggestions.
- Returns up to 10 suggestions, ordered by the number of mutual followers.

If the number of suggestions is insufficient, the fourth method is used to:

- Find random users or wineries not already followed by the user.
- Filter out the user themselves.
- Return a random subset up to the remaining number of slots needed.

This strategy ensures that the user is offered relevant and varied suggestions to expand their network, enhancing the social aspect of the application. The following figures show some of the code snippets involved in the process.

```
@GetMapping("/suggestions")
@Secured({ "ROLE_ADMIN", "ROLE_USER" })
@PreAuthorize("#username == authentication.principal.username or hasRole('ROLE_ADMIN')")
public List<Map<String, Object>> getSuggestedFollows(@RequestParam String username) {
    return userService.getSuggestedFollows(username);
}
```

Figure 4.44: `@RestController` method handling the get follow suggestions request.

```
public List<Map<String, Object>> getSuggestedFollows(String username) {
    List<Map<String, Object>> suggestions = userNeo4jRepository.getSuggestedFollows(username);

    if (suggestions.size() < 10) {
        int remaining = 10 - suggestions.size();
        List<Map<String, Object>> randoms = userNeo4jRepository.getRandomFollows(username, remaining);
        suggestions.addAll(randoms);
    }

    return suggestions;
}
```

Figure 4.45: Service method to get follow suggestions.

```
public List<Map<String, Object>> getSuggestedFollows(String username) {
    String query = """
        MATCH (me:User {username: $username})-[:FOLLOWS]->(friend:User)
        MATCH (friend)-[:FOLLOWS]->(suggested)
        WHERE NOT (me)-[:FOLLOWS]->(suggested) AND suggested <> me
            AND (suggested:User OR suggested:Winery)
        RETURN suggested.username AS username,
            suggested.thumbnail AS thumbnail,
            COUNT(*) AS mutualFollows
        ORDER BY mutualFollows DESC
        LIMIT 10
    """;

    List<Map<String, Object>> result = neo4jClient.query(query)
        .bind(username).to(name:"username")
        .fetch()
        .all()
        .stream()
        .map(record -> {
            Map<String, Object> r = new HashMap<>();
            r.put(key:"username", (String) record.get(key:"username"));
            r.put(key:"thumbnail", (String) record.get(key:"thumbnail"));
            r.put(key:"mutualFollows", ((Number) record.get(key:"mutualFollows")).intValue());
            return r;
        })
        .collect(Collectors.toList());

    System.out.println(">>> SUGGESTED: " + result);
    return result;
}
```

Figure 4.46: getSuggestedFollows() in UserNeo4jRepository

```
public List<Map<String, Object>> getRandomFollows(String username, int limit) {
    String query = """
        MATCH (s)
        WHERE (s:User OR s:Winery)
        MATCH (me:User {username: $username})
        WHERE NOT (me)-[:FOLLOWS]->(s) AND s.username <> $username
            AND s.username <> $username
        RETURN s.username AS username,
            s.thumbnail AS thumbnail
        ORDER BY rand()
        LIMIT $limit
    """;

    return neo4jClient.query(query)
        .bind(username).to(name:"username")
        .bind(limit).to(name:"limit")
        .fetch()
        .all()
        .stream()
        .map(record -> {
            Map<String, Object> r = new HashMap<>();
            r.put(key:"username", (String) record.get(key:"username"));
            r.put(key:"thumbnail", (String) record.get(key:"thumbnail"));
            return r;
        })
        .collect(Collectors.toList());
}
```

Figure 4.47: getRandomFollows() in UserNeo4jRepository

Follow a user

To follow another user or winery, the application must call the following methods:

- `public ResponseEntity<?> follow(@PathVariable String username, @PathVariable String targetUsername)`
- `public void follow(String followerUsername, String targetUsername)`
- `public void follow(String followerUsername, String targetUsername, boolean isWinery)`

The first method is a `@RestController` endpoint that handles a PUT request to the `/username/follow/targetUsername` path. It is secured so that only the user themselves can perform the follow operation, ensuring proper access control. The controller delegates the operation to the service layer, which determines whether the target is a user or a winery by querying the appropriate repository. If the target is valid, the service invokes the correct database logic; otherwise, it throws a `ResourceNotFoundException`. The third method interacts directly with the Neo4j graph. It performs the following steps:

- Matches the follower node in the graph.
- Matches the target node, which can be either a user or a winery depending on the `isWinery` flag.
- Creates a `FOLLOWS` relationship between the two nodes using the `MERGE` clause to avoid duplicates.

This structure ensures that users can establish social connections within the platform in a consistent and secure manner, enabling the application to maintain accurate and meaningful relationship data in the graph database. The following figures show some of the code snippets involved in the process.

```
@PostMapping("/{username}/follow/{targetUsername}")
@PreAuthorize("hasRole('ROLE_USER') and #username == authentication.principal.username")
public ResponseEntity<?> follow(
    @NotBlank(message = "Username cannot be blank.") @PathVariable String username,
    @NotBlank(message = "Target username cannot be blank.") @PathVariable String targetUsername) {
    userService.follow(username, targetUsername);
    return ResponseEntity.ok("Now following " + targetUsername);
}
```

Figure 4.48: `@RestController` method handling the follow request.

```

public void follow(String followerUsername, String targetUsername) throws ResourceNotFoundException {
    if (userRepository.existsByLogin_Username(targetUsername)) {
        userNeo4jRepository.follow(followerUsername, targetUsername, isWinery:false);
    } else if (wineryRepository.existsByLogin_Username(targetUsername)) {
        userNeo4jRepository.follow(followerUsername, targetUsername, isWinery:true);
    } else {
        throw new ResourceNotFoundException("No user or winery found with username: " + targetUsername);
    }
}

```

Figure 4.49: Service method to follow a user.

```

public void follow(String followerUsername, String targetUsername, boolean isWinery) {
    String query = isWinery
        ? """
            MATCH (f:User {username: $follower})
            MATCH (w:Winery {username: $target})
            MERGE (f)-[:FOLLOWS]->(w)
            """
        : """
            MATCH (f:User {username: $follower})
            MATCH (t:User {username: $target})
            MERGE (f)-[:FOLLOWS]->(t)
            """;
    neo4jClient.query(query)
        .bind(followerUsername).to(name:"follower")
        .bind(targetUsername).to(name:"target")
        .run();
}

```

Figure 4.50: follow() method in UserNeo4jRepository

4.5 Admin Operations

Admin operations on the reviews collection

An administrator (ROLE_ADMIN) has access to the following operations on the `reviews` collection:

- Retrieve all reviews in the system, paginated (GET /api/reviews).
- Retrieve a personalized review feed for any user (GET /api/reviews/feed).
- Delete all reviews in the system (DELETE /api/reviews).
- Delete a specific review by its ID (DELETE /api/reviews/{id}).
- Delete all reviews written by a specific user (DELETE /api/reviews/user/{username}).
- Delete all reviews associated with a specific wine (DELETE /api/reviews/wines/{wineId}).
- Delete all reviews associated with a specific vintage of a wine (DELETE /api/reviews/wines/{wineId}/vintages{vintageYear}).

Admin operations on the wines collection

An administrator (ROLE_ADMIN) has access to the following operations on the **wines** collection:

- Retrieve all wines in the system, paginated (GET /api/wines).
- Delete all wines in the system (DELETE /api/wines).
- Delete a specific wine by its ID (DELETE /api/wines/{id}).

Admin operations on the users collection

An administrator (ROLE_ADMIN) has access to the following operations on the **users** collection:

- Retrieve all users in the system, paginated (GET /api/users).
- Search users by first name, last name, or both (GET /api/users/search).
- Retrieve a specific user by their username (GET /api/users/{username}).
- Retrieve suggested follows for a specific user (GET /api/users/suggestions?username={username}).
- Update any user's information (PUT /api/users/{username}).
- Update any user's username (PUT /api/users/{username}/username/update?newUsername={newUsername}).
- Delete all users in the system (DELETE /api/users).
- Delete a specific user by their username (DELETE /api/users/{username}).

Admin operations on the wineries collection

An administrator (ROLE_ADMIN) has access to the following operations on the **wineries** collection:

- Retrieve all wineries in the system, paginated (GET /api/wineries).
- Search wineries by name, paginated (GET /api/wineries/search?name={name}).
- Retrieve a specific winery by username (GET /api/wineries/{username}).
- Update a winery's data (PUT /api/wineries/{username}).
- Update a winery's username (PUT /api/wineries/{username}/username/update).
- Add an image to a winery profile (PUT /api/wineries/{username}/addImage).

- Remove an image from a winery profile (PUT /api/wineries/{username}/removeImage).
- Delete all wineries in the system (DELETE /api/wineries).
- Delete a specific winery by username (DELETE /api/wineries/{username}).

Admin operations on the countries collection

An administrator (ROLE_ADMIN) has access to the following operations on the **countries** collection:

- Create a new country (POST /api/countries).
- Retrieve all countries in the system, paginated (GET /api/countries).
- Retrieve a specific country by name (GET /api/countries/{name}).
- Update a country's data (PUT /api/countries/{name}).
- Update a country's name (PUT /api/countries/{name}/name/update).
- Delete all countries in the system (DELETE /api/countries).
- Delete a specific country by name (DELETE /api/countries/{name}).

Admin operations on the regions collection

An administrator (ROLE_ADMIN) has access to the following operations on the **regions** collection:

- Create a new region associated with a country (POST /api/regions).
- Retrieve all regions in the system, paginated (GET /api/regions).
- Retrieve a specific region by name (GET /api/regions/{name}).
- Retrieve all regions in a specific country, paginated (GET /api/regions/countries/{country}).
- Update a region's associated country (PUT /api/regions).
- Delete all regions in the system (DELETE /api/regions).
- Delete a specific region by name (DELETE /api/regions/{name}).
- Delete all regions in a specific country (DELETE /api/regions/countries/{country}).

Admin operations on the styles collection

An administrator (ROLE_ADMIN) has access to the following operations on the `styles` collection:

- Create a new style (POST `/api/styles`).
- Retrieve all styles in the system, paginated (GET `/api/styles`).
- Retrieve a specific style by name (GET `/api/styles/{name}`).
- Update an existing style (PUT `/api/styles`).
- Delete all styles in the system (DELETE `/api/styles`).
- Delete a specific style by name (DELETE `/api/styles/{name}`).

Admin operations on analytics

An administrator (ROLE_ADMIN) has access to the following operations on analytics rankings:

- Delete all documents in the ranking `top-vintages-our-qop-type` (DELETE `/api/analytics/top-vintages-our-qop-type`).
- Delete a specific document by type in the ranking `top-vintages-our-qop-type` (DELETE `/api/analytics/top-vintages-our-qop-type/{type}`).
- Delete all documents in the ranking `top-vintages-qop-type` (DELETE `/api/analytics/top-vintages-qop-type`).
- Delete a specific document by type in the ranking `top-vintages-qop-type` (DELETE `/api/analytics/top-vintages-qop-type/{type}`).
- Delete all documents in the ranking `top-vintages-ratings-type` (DELETE `/api/analytics/top-vintages-ratings-type`).
- Delete a specific document by type in the ranking `top-vintages-ratings-type` (DELETE `/api/analytics/top-vintages-ratings-type/{type}`).
- Delete all documents in the ranking `top-wines-ratings-type` (DELETE `/api/analytics/top-wines-ratings-type`).
- Delete a specific document by type in the ranking `top-wines-ratings-type` (DELETE `/api/analytics/top-wines-ratings-type/{type}`).
- Delete all documents in the ranking `top-wineries-ratings` (DELETE `/api/analytics/top-wineries-ratings`).
- Delete a specific document by winery username in the ranking `top-wineries-ratings` (DELETE `/api/analytics/top-wineries-ratings/{winery_username}`).

Admin operations on the admins collection

An administrator (ROLE_ADMIN) has access to the following operations on the `admins` collection:

- Create a new admin (POST `/api/admins`).
- Retrieve all admins, paginated (GET `/api/admins`).
- Retrieve a specific admin by username (GET `/api/admins/{username}`).
- Update the admin's own data (PUT `/api/admins/{username}`).
- Update the admin's own username (PUT `/api/admins/{username}/username/update`).
- Update the admin's own password (PUT `/api/admins/{username}/password/update`).
- Delete the admin's own account (DELETE `/api/admins/{username}`).

Chapter 5

Database Choices

5.1 Introduction to VM Organization

Before exploring the specifics of this chapter, it is helpful to present an infographic that depicts the overall structure of the virtual machines (VMs) available to us, accompanied by explanations of the processes operating on each one.

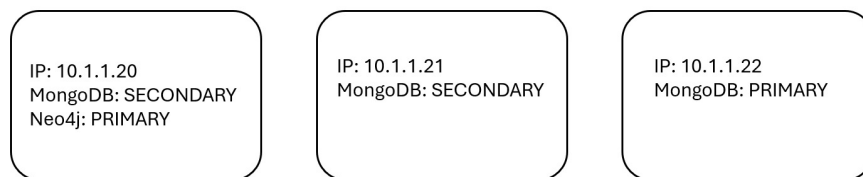


Figure 5.1: An overview of our VMs and their roles in the application.

5.2 Indexes

The indexes treated in the following subsections are indexes used to improve performance. In MongoDB, we also have other indexes that are mandatory for making the database fully functional. In particular, we need them for the aggregation pipelines to correctly insert the results into the respective collections. These indexes are the following:

5.2.1 MongoDB - reviews: `index(wine_id.id ASC)`

We need this index for the aggregation pipeline that updates the statistics on wines (average rating and rating counter). As shown below, the aggregation is quite intensive, due to the presence of a lookup on the collection *wines*. The index reduces the execution time and takes to 1 the number of collection scans necessary.

```
"totalDocsExamined": 84260808,  
"totalKeysExamined": 0,  
"collectionScans": 4825,  
"collectionSeeks": 0,  
"indexScans": 0,  
"indexSeeks": 0,  
"indexesUsed": [],
```

Figure 5.2: Collection scans without the index.

```
"totalDocsExamined": 22290,  
"totalKeysExamined": 17466,  
"collectionScans": 1,  
"collectionSeeks": 17466,  
"indexScans": 0,  
"indexSeeks": 4824,  
"indexesUsed": [  
  "index_for_wine_statistics_update"  
],
```

Figure 5.3: Collection scans with the index.

 **4824** documents returned

 **84260808** documents examined

 **64045 ms** execution time

Figure 5.4: Time needed without the index.

 **4824** documents returned

 **22290** documents examined

 **282 ms** execution time

Figure 5.5: Time needed with the index.

5.2.2 MongoDB - reviews: index(user_id.username ASC, rating DESC, created_at DESC)

We need this index for the aggregation pipelines that update the wine tips for each user and the tips for the wines for each user. As shown below, the aggregation is quite intensive, due to the presence of a lookup on the collection *reviews*. The index greatly reduces execution time and takes to 0 the number of collection scans necessary. The stats shown are the same for both the pipelines.

```
{
  "$lookup": {...},
  "totalDocsExamined": 87330000,
  "totalKeysExamined": 0,
  "collectionScans": 5000,
  "indexesUsed": [],
  "nReturned": 5000,
  "executionTimeMillisEstimate": 256496
},
```

Figure 5.6: Collection scans without the index.

```
{
  "$lookup": {...},
  "totalDocsExamined": 12496,
  "totalKeysExamined": 12496,
  "collectionScans": 0,
  "indexesUsed": [
    "index_for_wine_tips_and_new_wine_tips"
  ],
  "nReturned": 5000,
  "executionTimeMillisEstimate": 1748
},
```

Figure 5.7: Collection scans without the index.

 **4841** documents returned

 **5000** documents examined

 **460900 ms** execution time

Figure 5.8: Time needed with the index.

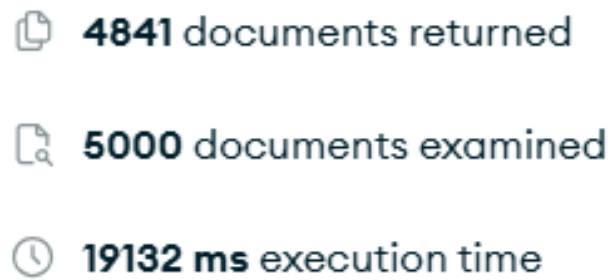


Figure 5.9: Time needed with the index.

5.2.3 MongoDB - wines: index(style.name ASC, statistics.ratings_average DESC, statistics.ratings_count DESC)

We need this index for the aggregation pipeline that update the wine tips for each user. The index is needed to improve the performance of another lookup. Below are shown the stats of the collection scans of this lookup. The stats regarding the execution time are in the previous sub-sections and regard the entire pipeline (with/without all the indexes used).

```
{
  "$lookup": {...},
  "totalDocsExamined": 57434544,
  "totalKeysExamined": 0,
  "collectionScans": 11906,
  "indexesUsed": [],
  "nReturned": 294178,
  "executionTimeMillisEstimate": 459125
},
```

Figure 5.10: Collection scans without the index.

```
{
  "$lookup": {...},
  "totalDocsExamined": 2693703,
  "totalKeysExamined": 281703,
  "collectionScans": 500,
  "indexesUsed": ["index_for_wine_tips"],
  "nReturned": 294178,
  "executionTimeMillisEstimate": 18072
},
```

Figure 5.11: Collection scans without the index.

5.2.4 MongoDB - wines: `index(style.name ASC, created_at DESC)`

We need this index for the aggregation pipeline that update the tips for new wines for each user. The index brings the exact same improving of the index at the previous sub-section.

5.2.5 Neo4j indexes considerations

We do not need mandatory Neo4j indexes for our graph to be fully functional. Considerations for indexes to improve graph performances have not been done.

5.3 Replication

5.3.1 MongoDB Replication

To meet the non-functional requirement of high availability, we have set up a replica set using the virtual machines available to us.

The organization of the replica set can be summarized as follows:

- A primary node with the IP address 10.1.1.22, responsible for receiving all write requests.
- Two secondary nodes with IP addresses 10.1.1.21 and 10.1.1.20, storing copies of the primary and receiving read requests.

We have used the next priorities to set up the replica set:

- 10.1.1.22: `priority=5`.
- 10.1.1.21: `priority=2`.
- 10.1.1.20: `priority=1`.

We have set the JAVA application to contact the cluster with the next settings:
`mongodb://10.1.1.22:27018,10.1.1.21:27018,10.1.1.20:27018/
?replicaSet=wineadvisor_repl_set&retryWrites=true&w=1
&readPreference=nearest`

5.3.2 Consideration on CAP Theorem

Looking at the requirements, it turns out that a strict consistency between replicas is not necessary: our application stays in the AP side of the CAP triangle. This permit us to set the parameters `w=1` and `readPreference=nearest`, with the conclusions we shared previously.

In the architectural design of our application, we have deliberately chosen to prioritize Availability and Partition Tolerance (AP) as per the CAP theorem. This decision is critical for an application like ours, which heavily relies on social networking features. By focusing on AP, we ensure that our system remains operational and accessible even in the event of network failures or data partitioning issues. This approach is vital to maintain uninterrupted user engagement with the social aspects of the application. We do this by replicating data across multiple nodes, thus ensuring availability even if some nodes are unreachable. While this might lead to temporary inconsistencies, it's a trade-off we accept to guarantee constant availability, which is crucial for user interactions within our social platform.

The configuration of the secondary nodes is aligned with our considerations. In the event of the primary node's failure, the secondary node at 10.1.1.21, with a priority of 2, is more likely to take over as the new primary node. This ensures continuous service delivery, even during primary node down times. The 10.1.1.21 has been chosen over the 10.1.1.20 due to the fact that the latter hosts the Neo4j graph (and process), to achieve a better load balancing and to reduce at the minimum point the problem of having a single point of failure.

5.4 Sharding

At present, the system does not require sharding, due to the small dataset size and good performance metrics during development and testing. However, since the application behaves similarly to a social network—with users following others and generating content—it is reasonable to anticipate that in large-scale production environments, sharding may become beneficial to ensure:

- **Scalability:** Distribute data across multiple servers to handle growing user volumes.
- **Performance:** Improve response times by reducing query bottlenecks.
- **Maintainability:** Better distribute workload and isolate data domains for easier updates and monitoring.

Possible Sharding Strategies

In this context, the following strategies are under consideration for potential future scaling:

- **User-based sharding:**
Data is partitioned based on user identity (e.g., by username). This approach is effective when most operations are scoped to individual users, such as personal reviews, likes, and follows.

- **Geographic sharding:**

Users (and their associated content) are grouped by country or region. This is well-suited if the application grows with a user base distributed in distinct countries and can help reduce latency by locating data closer to the user.

- **Feature-based sharding:**

Functional domains are separated across different shards—for example, storing reviews in one shard, users in another, and wineries in a third. This approach helps optimize domain-specific queries and isolate workloads.

- **Hybrid sharding:**

A combination of multiple strategies, such as applying user-based sharding within geographic partitions. While this adds complexity, it allows for more balanced and flexible scaling based on real-world usage patterns.